DOCUMENTATION SECTION

ACTUS Organization

The ACTUS Financial Research Foundation — the open standard for financial contract representation, its taxonomy, data dictionary, technical specification, and the organizations behind it.

- ACTUS Standard Overview
- Contract Type Taxonomy — ~32 Types
- Data Dictionary — CT Attributes & Applicability
- Technical Specification & State Transition Functions
- Business Rules Notation
- Developer Access & Reference Implementation
- Membership & Resources

Table of Contents

ACTUS — Algorithmic Contract Types Unified Standards

About ACTUS

Fundamentals

Taxonomy

Applicability

Data Dictionary

Technical Specification

Rules

Developers

Membership

Resources

News

- ACTUS Technical Specifications Java
- ACTUS Technical Specifications
- Package Structure
- Formal Framework
- Public API
- State Transition Functions and Payoff Functions
- Contract Type Implementations
- Schedule Generation
- Contract Events
- Contract Types in the Specification

Payoff and State Transition Functions

Repository

State Space

Conventions

Schedule Generation

Type System

Test Infrastructure

ACTUS Org

ACTUS — Algorithmic Contract Types Unified Standards

> *This is the current documentation, now improved with a new style and technical structure. It demonstrates how Markdown can be used as a technology to make website updates easier to maintain and manage.*

ACTUS — Algorithmic Contract Types Unified Standards

ACTUS — the Algorithmic Contract Types Unified Standards — is a global initiative to bring clarity, consistency, and precision to the way financial contracts are represented and analyzed.

The Standard

The goal of ACTUS is to provide open, royalty-free standards for the combined data and algorithmic representation of financial contracts in order to promote efforts for transparent interoperability of financial contract information as well as support the use of the ACTUS standards and open source software in developing a broad range of applications and systems that utilize the forward-looking cash flow analysis that ACTUS enables.

The ACTUS Financial Research Foundation provides a universal infrastructure layer with two fundamental components:

- **The ACTUS Data Dictionary** — a universal set of legal terms (contract type attributes) used as parameters throughout different financial agreements.
- **The ACTUS Cash Flow Generating Algorithms** — the logic embedded in legal agreements that eventually turns the contract terms into actual cash flows, or more generally business events.

Why Financial Contracts?

Financial contracts are basic building blocks for finance, underlying cash flow agreements, risk analysis, and nearly all financial reports. The centrality of expected cash flows for financial analysis is obvious and undisputed. Standardized contract term specification and cash flow generating algorithms provide a basis for all further financial analysis.

ACTUS is based on a modeling paradigm in which standardized Contract Types are the granular building blocks of the financial world, with a smart and machine-readable algorithmic representation of all legal agreements and a strict separation of the known from the unknown.

What ACTUS Covers

With very few exceptions, virtually all financial instruments currently known can be represented by parameterizing and combining a set of roughly 32 ACTUS financial contract types — ranging from very simple Principal at Maturity (PAM) or Annuity (ANN) bonds to Stocks (STK) or Commodity (COM) instruments to derivatives like futures (FUTUR) or options (OPTNS) and credit default swaps (CDSWP).

Who It Is For

Intended users of the ACTUS infrastructure layer encompass all participants in the financial system, from fund and portfolio managers, banks with traditional or digital infrastructure, clearing houses, and regulatory agencies.

ACTUS can be used to incorporate forward-looking cash flow analysis, risk management, or digital asset management capabilities in financial businesses. You can focus on the creation and composition of your financial products, use ACTUS to generate detailed future cash flow projections for your product mix using scenarios of your choice, and then feed the forecasted cash flow results into analytics for valuation, liquidity, and risk management.

Recognition

In October 2021, ACTUS was added as the second reference after ISO 20022 to a database run by the Office of Financial Research (OFR), an arm of the US Treasury.

Contents of This Section

- About — Organization, governance, and mission
- Fundamentals — The ACTUS modeling paradigm

- Taxonomy — Classification of contract types
- Applicability — Use cases across the financial system
- Data Dictionary — Contract attributes and permitted values
- Technical Specification — The formal algorithmic standard
- Rules — Business rules for attribute applicability
- Developers — Reference implementation and GitHub access
- Membership — Joining the ACTUS Users Association
- Resources — Downloads, demo apps, and documentation
- News — Conferences, webinars, and announcements

About ACTUS

About ACTUS

Mission

The goal of ACTUS is to provide open, royalty-free standards for the combined data and algorithmic representation of financial contracts in order to promote efforts for transparent interoperability of financial contract information as well as support the use of the ACTUS standards and open source software in developing a broad range of applications and systems that utilize the forward-looking cash flow analysis that ACTUS enables.

The Two Organizations

Project ACTUS has been incorporated in the form of two not-for-profit corporations to hold the intellectual property of ACTUS and to further the development as well as maintenance of the ACTUS Contract Types and their use.

ACTUS Financial Research Foundation

The ACTUS Financial Research Foundation is a 501(c)(3) not-for-profit research corporation. It is the holder and curator of the intellectual property of the royalty-free, open standard that is ACTUS.

The primary purpose of the ACTUS Financial Research Foundation is to conduct scientific research relating to the creation of a data standard for financial contracts that would provide better regulatory and risk management information.

ACTUS Users Association

The ACTUS Users Association is a 501(c)(6) not-for-profit trade association. It is primarily concerned with building up and further developing an active community and industry support.

The ACTUS Users Association develops, maintains, and releases:

- The ACTUS Technical Specification
- A reference implementation in Java through GitHub under the ACTUS Core License

What ACTUS Provides

The adoption of the ACTUS standard provides a concise data dictionary that defines the terms present in a particular type of financial contract, a simple but complete taxonomy of the fundamental algorithmic contract type patterns, and reference code that calculates the cash flow obligations established by the contract so they can be accurately projected, analyzed, and acknowledged by all parties over the life of the contract.

Recognition

ACTUS is regarded as important for transaction processing, risk management, financial regulation, the tokenization of financial instruments, and the development of smart contracts for decentralized finance (DeFi) using blockchain technology.

In October 2021, ACTUS was added as the second reference after ISO 20022 to a database run by the Office of Financial Research (OFR), an arm of the US Treasury.

Fundamentals

Fundamentals

The Modeling Paradigm

ACTUS is based on a modeling paradigm in which standardized Contract Types (CTs) are the granular building blocks of the financial world. ACTUS aims to raise this standardization to a universal and global level with a smart and machine-readable algorithmic representation of all legal agreements and a strict separation of the known from the unknown.

Known and Unknown

The only knowns are the legal agreements and possibly the current state of the risk factors. The unknowns are the future states of the risk factors — in particular, the market, counterparty, and behavioral risks.

Given the state of contracts and a scenario of risk factors, it is possible to derive the contract events, which generate the expected state-contingent cash flows. From expected state-contingent cash flows it is possible to derive the analytical metrics of interest, such as income, value, and liquidity.

The Two Standards

The Data Standard

The Data Standard defines a universal set of legal terms — or CT Attributes — used as parameters throughout the different financial agreements. It is implemented in the form of a Data Dictionary with attribute applicability by contract type.

The Algorithmic Standard

The Algorithmic Standard defines the logic embedded in legal agreements that eventually turns the contract terms into actual cash flows, or more generally business events. The formal specification of this standard is documented in the Technical Specification.

The State Machine Model

The financial contract is modeled as a persistable state machine which follows a possibly dynamic event schedule and keeps track of the current state and ownership. The formal four-component definition of this model — states, events, transitions, and payoffs — is given in the Technical Specification Framework.

Risk Factors

Financial contracts and current states of the risk factors — yield curves, FX rates, and similar market data — are the core inputs. Given the state of contracts and a scenario of risk factors, it is possible to derive the contract events which generate expected state-contingent cash flows.

Applications

The two standards together support forward-looking financial and risk analysis. ACTUS can form the computation core of smart financial contracts, as well as a broad range of financial activities, including:

- Transaction processing
- Risk management
- Liquidity analysis
- Pre-trade analysis
- Forward business planning
- Building accounts

Taxonomy

Taxonomy

Overview

The ACTUS taxonomy shows the classification of financial contracts as defined by ACTUS. Contract Types are defined based on the underlying contractual algorithm patterns that respectively cover different classes of financial products that each contract type pattern is able to express.

With very few exceptions, virtually all financial instruments currently known can be represented by parameterizing and combining a set of roughly 32 ACTUS financial contract types — ranging from very simple Principal at Maturity (PAM) or Annuity (ANN) bonds to Stocks (STK) or Commodity (COM) instruments to derivatives like futures (FUTUR) or options (OPTNS) and credit default swaps (CDSWP).

Basic: Fixed Income — With Maturities

These contract types model instruments where principal and interest are exchanged according to a defined schedule.

Code	Name	Description
PAM	Principal at Maturity	A bullet loan where principal is repaid in full at maturity. Periodic interest payments are made during the life of the contract.
ANN	Annuity	A loan amortized in the annuity method with a series of payments made between equal time intervals in equal amounts. The sum of interest and principal components is fixed, but the individual amounts change with each payment.
NAM	Negative Amortizer	Periodic payments of a fixed amount that may not cover the full interest due. Unpaid accrued interest is capitalized into the outstanding principal balance, causing the loan balance to increase over time.
LAM	Linear Amortizer	Periodic repayments of fixed principal amounts. The interest payment is recalculated with each repayment of principal, decreasing over time as the principal reduces.

Code	Name	Description
ANX	Exotic Annuity	Step-up variation of the Annuity (ANN) contract type.
NAX	Exotic Negative Amortizer	Step-up variation of the Negative Amortizer (NAM) contract type.
LAX	Exotic Linear Amortizer	Step-up variation of the Linear Amortizer (LAM) contract type.
CLM	Call Money	An instrument where the client has the option to call the contract with a notice period after the current date. The contract will not be settled until the client exercises this option.
PBN	Perpetual Bond	A bond with no maturity date that pays periodic interest indefinitely.

Basic: Fixed Income — Without Maturities

Code	Name	Description
CSH	Cash	Represents a cash position.
UMP	Undefined Maturity Profile	A contract type where the client has the option to call the contract with a notice period. Similar to CLM but with an undefined maturity profile.

Basic: Index-Based

Code	Name	Description
STK	Stock	Represents equity holdings — shares of stock in a company.
COM	Commodity	Physical contracts holding a number of underlying units of a specific good, such as a number of barrels of oil.

Combined Contracts: Symmetric (Derivatives)

Code	Name	Description
SWAPS		

Code	Name	Description
	Plain Vanilla Interest Rate Swap	Exchange of fixed and floating interest rate payment streams between counterparties.
SWPPV	Plain Vanilla IR Swap PV	Present value variant of the plain vanilla interest rate swap.
FXOUT	Foreign Exchange Outright	An agreement to exchange one currency for another at a specified future date and a rate fixed at the time of agreement.
FUTUR	Future	A standardized contract to buy or sell an asset at a predetermined price at a specified future date.

Combined Contracts: Asymmetric (Options)

Code	Name	Description
OPTNS	Option	The right, but not the obligation, to buy or sell an underlying asset at a predetermined price on or before a specified date.
CAPFL	Caplet / Floorlet	An interest rate cap or floor component that limits the maximum or minimum interest rate applicable in a period.
BNDCP	Bond with Embedded Call/ Put	A bond that includes an embedded option allowing early redemption by either the issuer or the holder.
BNDWR	Bond with Warrant	A bond that includes a warrant giving the holder the right to purchase equity at a specified price.
EXOTI	Exotic Option	A non-standard option with complex features that differ from conventional options.

Securitization

Code	Name	Description
CDSWP	Credit Default Swap	Transfers the credit exposure of fixed income products between parties. Provides the buyer protection against a credit event.
TRSWP		

Code	Name	Description
	Total Return Swap	A swap agreement in which one party makes payments based on a set rate while the other makes payments based on the total return of an underlying asset.
CLNTE	Credit Linked Note	A structured note with an embedded credit default swap.
SCRMR	Securitization Market Risk	A securitization vehicle with market risk exposure.
SCRCR	Securitization Credit Risk	A securitization vehicle with credit risk exposure.

Credit Enhancement

Code	Name	Description
CEG	Guarantee	Creates a relationship between a guarantor, an obligee, and a debtor, moving the exposure from the debtor to the guarantor.
CEC	Collateral	Creates a relationship between collateral, an obligee, and a debtor, covering the exposure from the debtor with the collateral.
BCS	Basic Credit Support	A basic credit support structure.

Applicability

Applicability

Intended Users

Intended users of the ACTUS infrastructure layer encompass all participants in the financial system, from fund and portfolio managers, banks with traditional or digital infrastructure, clearing houses, and regulatory agencies.

How ACTUS Is Used

ACTUS can be used to incorporate forward-looking cash flow analysis, risk management, or digital asset management capabilities in financial businesses. You can focus on the creation and composition of your financial products, use ACTUS to generate detailed future cash flow projections for your product mix using scenarios of your choice, and then feed the forecasted cash flow results into analytics for valuation, liquidity, and risk management.

Financial Activities

ACTUS can form the computation core of smart financial contracts, as well as a broad range of financial activities, including:

- Transaction processing
- Risk management
- Liquidity analysis
- Pre-trade analysis
- Forward business planning
- Building accounts

Analytics

The two standards together support forward-looking financial and risk analysis, encompassing transaction processing, accounting, risk management, and regulatory reporting. From expected state-contingent cash flows it is possible to derive the analytical metrics of interest, such as income, value, and liquidity.

Regulatory Applications

ACTUS can be used to improve the quality and value of the data collected by regulators and, at the same time, reduce the regulatory reporting burden on banks.

Digital Assets

ACTUS is also applicable for real-world financial use cases in blockchain-based financial applications. The algorithms also constitute the core computations required for smart financial contracts.

Competition and Applications

The ACTUS Foundation organizes a competition for building prototype applications that use ACTUS. The challenge for competitors is to provide services or applications utilizing ACTUS to provide increased value for financial institutions and other users with critical reliance on financial contracts.

Examples of such services include programs that increase the ease of incorporation of ACTUS into existing systems as well as applications enabling special types of analysis, such as the field of risk analysis. Both commercial proprietary and open-source applications are welcome.

Data Dictionary

Data Dictionary

Overview

The data dictionary provides a collection of all contract attributes as defined by the ACTUS Standard. The Data Standard defines a universal set of legal terms — or CT Attributes — used as parameters throughout the different financial agreements. It is implemented in the form of a Data Dictionary with Attribute Applicability by Contract Type.

Permitted values for each term are listed in the ACTUS data dictionary.

Available Formats

The dictionary comes in two forms:

- **Excel** — for human readability
- **JSON** — for programmatic integration

The JSON dictionary (`actus-dictionary.json`) is derived from the Excel dictionary (`actus-dictionary.xlsx`) through an R script.

GitHub Repository

The data dictionary is maintained in the GitHub repository at `actusfrf/actus-dictionary`. The repository contains the ACTUS dictionary and generation scripts.

Files available in the repository include:

- `actus-dictionary.xlsx` — the Excel format dictionary
- `actus-dictionary.json` — the JSON format dictionary
- `actus-dictionary-terms.json` — the dictionary terms in JSON format

Attribute Applicability

The data dictionary defines which contract attributes apply to which contract types. Business rules in the dictionary specify whether each attribute is mandatory, optional, or non-applicable for a given contract type.

Technical Specification

Technical Specification

The Algorithmic Standard defines the logic embedded in legal agreements that eventually turns the contract terms into actual cash flows, or more generally business events. The Technical Specification is developed, maintained, and released by the ACTUS Users Association.

The specification is a single LaTeX source file (`actus-techspecs.tex`) in the GitHub repository `actusfrf/actus-techspecs`. The compiled PDF is available at `https://www.actusfrf.org/algorithmic-standard`. The document is licensed under Creative Commons Attribution Share-Alike (CC-BY-SA) version 4.0.

The ACTUS Users Association also releases a Java reference implementation through GitHub under the ACTUS Core License, consistent with this specification.

Detailed Documentation

The full content of the `actusfrf/actus-techspecs` repository has been converted into the following documents:

- Overview — document purpose, version, license, and contents
- Formal Framework — the state machine model: states, events, transitions, payoffs, inputs, outputs
- STF and POF Functions — the two function families, event type table, naming conventions
- Schedule Generation — cycle notation, date adjustment, event ordering
- Contract Types — all ~32 contract types and what the spec defines per type
- Repository — file listing, build instructions, contributing

Rules

Rules

Overview

Business rules define the applicability of contract attributes to the different contract types. Each rule specifies whether a given contract attribute (CA) is non-applicable, mandatory, or optional for a given contract type.

Rule Notation

A business rule is written in the form `a(b,c,d)`, where each component has the following meaning:

- **Rule** `a` — defines whether a contract attribute is non-applicable, mandatory, or optional for the contract type.
- **Rule** `b` — indicates whether the same applicability rule is subject to a further conditional group rule `c`.
- **Rule** `c` — the conditional group rule referenced by `b`.
- **Rule** `d` — defines an additional applicability rule that may apply in the context of stand-alone or parent-child contract type relationships.

Applicability Designations

Each contract attribute is assigned one of the following applicability designations for each contract type:

- **Mandatory** — the attribute must be provided for the contract type.
- **Optional** — the attribute may be provided but is not required.
- **Non-applicable** — the attribute does not apply to the contract type and should not be provided.

Rule `e`

Rule `e` defines additional constraints for array-type contract attributes, specifying the structure and permitted content of attribute arrays.

Attribute Applicability Matrix

The full attribute applicability matrix is contained in the ACTUS data dictionary, which lists the applicability designation for every combination of contract attribute and contract type.

Developers

Developers

Registration

All parties who wish to access the ACTUS GitHub open-source repository are invited to register on the ACTUS website. This registration requirement enables ACTUS to comply with regulatory requirements.

Reference Implementation

The ACTUS Users Association develops, maintains, and releases a reference implementation in Java through GitHub under the ACTUS Core License.

The Java reference implementation includes the actus-core library, which is the core library implementing the Contract Types. The actus libraries constitute the algorithmic standards as Java libraries.

ACTUS Core License

The reference implementation is distributed as FOSS software under the ACTUS Core License. The license terms include restrictions on how modified software can be redistributed, in order to guarantee the integrity of the implemented contract logic with the ACTUS Technical Specification.

The ACTUS Core License is maintained in the GitHub repository `actusfrf/actus-core-license`.

GitHub Organization

The ACTUS Financial Research Foundation maintains a GitHub organization at `github.com/actusfrf`. Repositories include:

- `actus-core` — the core Java library implementing the Contract Types
- `actus-techspecs` — the ACTUS Technical Specifications in LaTeX
- `actus-dictionary` — the ACTUS data dictionary and generation scripts
- `actus-core-license` — the ACTUS Core License

- `actus-service` — the ACTUS service
- `actus-webapp` — the ACTUS Demo App web application

Membership

Membership

The ACTUS Users Association

The ACTUS Users Association is a 501(c)(6) not-for-profit trade association organized under the laws of the State of Delaware and granted not-for-profit status by the United States Internal Revenue Service.

The ACTUS Users Association is primarily concerned with building up and further developing an active community and industry support. It develops, maintains, and releases the ACTUS Technical Specification and a reference implementation in Java through GitHub under the ACTUS Core License.

Organizational Structure

Decision-making responsibility rests with the board of directors, relying on recommendations and significant input from the members of the Association.

How to Apply

To become a member of the ACTUS Users Association, fill out the membership application form. The board of directors will carefully review your application and inform you of the acceptance decision within twenty (20) business days.

Resources

Resources

Community Edition

The ACTUS Community Edition download and related demo apps make it easy to see how ACTUS can be used to incorporate forward-looking cash flow analysis, risk management, or digital asset management capabilities in a financial business.

Demo Application

The ACTUS Demo App is a lightweight web application showcasing the ACTUS standards. The demo can be accessed at `demo.actusfrf.org`. Opening that address in a browser brings up a list of contract types that can be explored interactively.

The source code of the ACTUS Demo App is available in the GitHub repository `actusfrf/actus-webapp`.

Quick Start

The ACTUS Quick Start provides a step-by-step guide to quickly set up and use the ACTUS framework using Docker. The Quick Start page provides an index to helpful documentation and instructions regarding where to find and access resources, demos, and downloadable ACTUS packages.

The Quick Start is available at `documentation.actusfrf.org/docs/quickstart`.

Documentation

Comprehensive documentation is available at `documentation.actusfrf.org`. The documentation includes guides on modeling risk factors, simulating contracts, and working with risk databases, as well as hands-on demonstrations and practical examples.

News

News

2024 ACTUS Conference

The ACTUS Foundation organized its annual conference. The conference featured presentations and panel discussions on the many ways that the ACTUS financial contract standards can contribute to solutions to challenges in the financial industry.

DaDFiR3 Conference

The DaDFiR3 Conference brought together 50+ attendees to discuss the DaDFiR3 Project, ACTUS, and their potential impact on the financial industry. Dr. Willi Brammertz held a presentation in which he elaborated on how ACTUS is set to improve regulation and bring substantial benefits to the financial industry.

Speakers at the conference included Magdalena Boškić (Sygnum Bank), Dr. Willi Brammertz (ARIADNE and ACTUS Financial Research Foundation), Philippe Brügger (FINMA), Mark A. Greenslade (Casper Association), Francis Gross (European Central Bank), Moritz Karber (European Central Bank), Ralf Kubli (Casper Association), and Eva Selamlar-Leuthold (Swiss Financial Innovation Desk).

Webinar: The New World of Finance — Marlowe

Dr. Willi Brammertz presented a webinar titled "The New World of Finance" covering ACTUS and the Marlowe smart contract language.

ACTUS Competition

The ACTUS Financial Research Foundation organizes a competition for building prototype applications that use ACTUS. The challenge for competitors is to provide services or applications utilizing ACTUS to provide increased value for financial institutions and other users with critical reliance on financial contracts.

Examples of such services include programs that increase the ease of incorporation of ACTUS into existing systems, as well as applications enabling special types of analysis,

such as risk analysis. Both commercial proprietary and open-source applications are welcome.

ACTUS Technical Specifications Java

ACTUS Java Reference Implementation

Purpose

The Java reference implementation is the canonical realisation of the ACTUS Technical Specification. It provides a complete, testable codebase that transforms ACTUS contract terms and risk factor observations into a schedule of typed, valued contract events. It is maintained by the ACTUS Users Association and released under the ACTUS Core License.

The implementation covers **19 contract types**, **28 event types**, and **200+ individual payoff and state transition functions**.

Entry Points

There are exactly two methods a caller needs to know:

```
// Step 1 — generate the event schedule up to a horizon
ArrayList<ContractEvent> schedule =
    ContractType.schedule(LocalDateTime to, ContractModelProvider model);

// Step 2 — evaluate each event against the current state and risk factor
model
ArrayList<ContractEvent> results =
    ContractType.apply(ArrayList<ContractEvent> events,
                       ContractModelProvider model,
                       RiskFactorModelProvider observer);
```

`ContractType` is the single public dispatcher. It inspects `model.getAs("contractType")` and routes both calls to the correct contract-type class.

Layered Architecture

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    CALLER["Caller"] --> API["ContractType\ndispatcher"]
    API --> IMPL["Contract Implementations\n(19 classes)"]
```

```
    IMPL --> EF["EventFactory\n(creates ContractEvent objects)"]
    EF --> CE["ContractEvent\n(schedule time · event type · POF · STF)"]
    CE --> POF["PayOffFunction\ninterface (200+ impls)"]
    CE --> STF["StateTransitionFunction\ninterface (200+ impls)"]
    POF --> SS["StateSpace\n(contract state vector)"]
    STF --> SS
    CE --> CONV["Conventions\n(DayCount · BusinessDay · EOM)"]
    IMPL --> SCHED["ScheduleFactory\n(date sequence generation)"]
    SCHED --> CA["CycleAdjuster\n(period / weekday)"]
    POF --> RFM["RiskFactorModelProvider\n(external market data)"]
    STF --> RFM
    POF --> CM["ContractModelProvider\n(contract terms)"]
    STF --> CM
```

Processing Flow

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
sequenceDiagram
    participant Caller
    participant CT as ContractType
    participant Impl as e.g. PrincipalAtMaturity
    participant EF as EventFactory
    participant CE as ContractEvent
    participant POF
    participant STF

    Caller->>CT: schedule(to, model)
    CT->>Impl: schedule(to, model)
    Impl->>EF: createEvent / createEvents(dates, type, currency, POF, STF, cont
    EF-->>Impl: ContractEvent list
    Impl-->>CT: sorted ArrayList<ContractEvent>
    CT-->>Caller: ArrayList<ContractEvent>

    Caller->>CT: apply(events, model, observer)
    CT->>Impl: apply(events, model, observer)
    Impl->>Impl: initialise StateSpace from model
    loop for each event
        Impl->>CE: eval(states, model, observer, dayCounter, timeAdjuster)
        CE->>POF: eval(...) → double payoff
        CE->>STF: eval(...) → StateSpace
        CE-->>Impl: updated states + payoff
    end
    Impl-->>Caller: ArrayList<ContractEvent> with states and payoffs
```

Package Summary

Package	Contents
`org.actus`	Three runtime exception classes
`org.actus.attributes`	`ContractModelProvider` interface + `ContractModel` implementation
`org.actus.contracts`	`ContractType` dispatcher + 19 contract type classes
`org.actus.events`	`ContractEvent`, `EventFactory`, `EventSequence`
`org.actus.functions`	`PayOffFunction` and `StateTransitionFunction` interfaces + 200+ implementations
`org.actus.states`	`StateSpace` — the contract state vector
`org.actus.conventions`	Business day, day count, end-of-month, contract role conventions
`org.actus.time`	`ScheduleFactory`, `CycleAdjuster`, calendars
`org.actus.types`	30+ enum and type classes
`org.actus.externals`	`RiskFactorModelProvider` interface
`org.actus.util`	`CommonUtils`, `CycleUtils`, `RedemptionUtils`, `StringUtils`, `Constants`

Codebase Metrics

Metric	Value
Main source files	198
Test files	45 + 7 utilities
Contract types supported	19
Event types	28
Function implementations (POF + STF)	200+
Test JSON data files	18
Maximum contract lifetime constant	50 years

Documentation Contents

- Package Structure — full package tree and module relationships
- Public API — `ContractType`, `ContractModelProvider`, `RiskFactorModelProvider`
- Contract Types — all 19 implementations and their schedule/apply logic
- Contract Events — `ContractEvent`, `EventFactory`, event ordering
- Functions — POF and STF interfaces, naming convention, complete inventory
- State Space — all state variables and how state evolves
- Conventions — day count, business day, EOM, contract role
- Schedule Generation — `ScheduleFactory`, `CycleAdjuster`, calendars
- Type System — all enums and value types
- Test Infrastructure — JSON test format, test utilities, test patterns

ACTUS Technical Specifications

ACTUS Technical Specifications

The ACTUS Technical Specification Document provides a formal specification of the ACTUS standard for the algorithmic representation of financial contracts. It is developed, maintained, and released by the ACTUS Users Association.

The source is a single LaTeX file (`actus-techspecs.tex`) in the GitHub repository `actusfrf/actus-techspecs`, Version 1.1 (June 8, 2020), licensed under Creative Commons Attribution Share-Alike (CC-BY-SA) 4.0. The compiled PDF is available at `https://www.actusfrf.org/algorithmic-standard`.

Contents

- Formal Framework — the four-component state machine model, inputs, outputs, and per-contract-type definition structure
- STF and POF Functions — the two function families, event type codes, and naming conventions
- Schedule Generation — cycle notation, date adjustment conventions, and event ordering
- Contract Types — all ~32 contract types and what the spec defines per type
- Repository — file listing, build instructions, license, contributing

Relationship to the Data Standard

The Algorithmic Standard and the Data Standard (Data Dictionary) are two sides of a single standard. Contract terms from the data dictionary are the inputs to the algorithms defined here.

Package Structure

Package Structure

Source Tree

```
actus-core/
├── main/java/org/actus/
│   ├── AttributeConversionException.java       ← thrown when attribute
cast fails
│   ├── ContractTypeUnknownException.java        ← thrown for
unrecognised CT code
│   ├── RiskFactorNotFoundException.java         ← thrown when risk
factor ID absent
│   │
│   ├── attributes/
│   │   ├── ContractModelProvider.java           ← interface: get
attribute by name
│   │   └── ContractModel.java                  ← map-backed
implementation (1035 lines)
│   │
│   ├── contracts/
│   │   ├── ContractType.java                   ← public dispatcher
(schedule + apply)
│   │   ├── PrincipalAtMaturity.java             ← PAM  (279 lines)
│   │   ├── LinearAmortizer.java                 ← LAM  (414 lines)
│   │   ├── NegativeAmortizer.java               ← NAM  (448 lines)
│   │   ├── Annuity.java                         ← ANN  (500 lines)
│   │   ├── ExoticLinearAmortizer.java           ← LAX  (523 lines)
│   │   ├── CallMoney.java                       ← CLM
│   │   ├── UndefinedMaturityProfile.java        ← UMP
│   │   ├── Cash.java                            ← CSH
│   │   ├── Stock.java                           ← STK
│   │   ├── Commodity.java                       ← COM
│   │   ├── ForeignExchangeOutright.java         ← FXOUT
│   │   ├── PlainVanillaInterestRateSwap.java     ← SWPPV
│   │   ├── Swap.java                            ← SWAPS
│   │   ├── CapFloor.java                        ← CAPFL
│   │   ├── Option.java                          ← OPTNS
│   │   ├── Future.java                          ← FUTUR
│   │   ├── CreditEnhancementGuarantee.java      ← CEG
│   │   ├── CreditEnhancementCollateral.java     ← CEC
```

```
|   |   └── BoundaryControlledSwitch.java        ← BCS
|   |
|   ├── events/
|   |   ├── ContractEvent.java                   ← single event (256
lines)
|   |   ├── EventFactory.java                    ← factory (114 lines)
|   |   └── EventSequence.java                   ← ordering offsets (115
lines)
|   |
|   ├── externals/
|   |   └── RiskFactorModelProvider.java         ← interface for market
data
|   |
|   ├── functions/
|   |   ├── PayOffFunction.java                  ← interface
|   |   ├── StateTransitionFunction.java         ← interface
|   |   ├── pam/   (27 POF_*/STF_* classes)
|   |   ├── lam/   (18 classes)
|   |   ├── nam/   ( 3 classes)
|   |   ├── lax/   ( 8 classes)
|   |   ├── ann/   ( 1 class)
|   |   ├── clm/   ( 3 classes)
|   |   ├── csh/   ( 1 class)
|   |   ├── stk/   ( 7 classes)
|   |   ├── fxout/ ( 8 classes)
|   |   ├── swppv/ (11 classes)
|   |   ├── swaps/ ( 5 classes)
|   |   ├── capfl/ ( 2 classes)
|   |   ├── optns/ ( 8 classes)
|   |   ├── futur/ ( 4 classes)
|   |   ├── ceg/   ( 9 classes)
|   |   ├── cec/   ( 3 classes)
|   |   ├── bcs/   ( 4 classes)
|   |   └── ump/   ( n classes)
|   |
|   ├── states/
|   |   └── StateSpace.java                     ← contract state vector
(109 lines)
|   |
|   ├── conventions/
|   |   ├── businessday/
|   |   |   ├── BusinessDayConvention.java       ← interface
|   |   |   ├── BusinessDayAdjuster.java         ← coordinator
|   |   |   ├── Following.java
|   |   |   ├── ModifiedFollowing.java
|   |   |   ├── Preceeding.java
|   |   |   ├── ModifiedPreceeding.java
```

```
│   │   │   ├── Same.java
│   │   │   ├── ShiftCalcConvention.java         ← interface
│   │   │   ├── CalcShift.java
│   │   │   └── ShiftCalc.java
│   │   ├── contractrole/
│   │   │   └── ContractRoleConvention.java      ← roleSign() utility
│   │   ├── daycount/
│   │   │   ├── DayCountConventionProvider.java  ← interface
│   │   │   ├── DayCountCalculator.java          ← factory + coordinator
│   │   │   ├── ActualThreeSixty.java
│   │   │   ├── ActualThreeSixtyFiveFixed.java
│   │   │   ├── ActualActualISDA.java
│   │   │   ├── ActualThreeThirtySix.java
│   │   │   ├── BusinessTwoFiftyTwo.java
│   │   │   ├── ThirtyEThreeSixty.java
│   │   │   ├── ThirtyEThreeSixtyISDA.java
│   │   │   └── TwentyEightThreeThirtySix.java
│   │   └── endofmonth/
│   │       ├── EndOfMonthConvention.java        ← interface
│   │       ├── EndOfMonthAdjuster.java          ← static adjust()
│   │       ├── EndOfMonth.java
│   │       └── SameDay.java
│   │
│   ├── time/
│   │   ├── ScheduleFactory.java                ← date sequence builder
(177 lines)
│   │   ├── CycleAdjuster.java                  ← period vs weekday
dispatcher
│   │   ├── CycleAdjusterProvider.java          ← interface
│   │   ├── PeriodCycleAdjuster.java            ← ISO 8601 period
arithmetic
│   │   ├── WeekdayCycleAdjuster.java           ← nth weekday of month
│   │   ├── TimeAdjuster.java                   ← sub-hour normalisation
│   │   └── calendar/
│   │       ├── BusinessDayCalendarProvider.java ← interface
│   │       ├── NoHolidaysCalendar.java          ← NC: all days are
business days
│   │       ├── MondayToFridayCalendar.java      ← MF: Mon–Fri
│   │       └── MondayToFridayWithHolidaysCalendar.java ← MFH: Mon–Fri
minus holidays
│   │
│   ├── types/
│   │   ├── ContractTypeEnum.java               ← 19 CT codes
│   │   ├── EventType.java                      ← 28 event type codes
│   │   ├── ContractRole.java                   ← 13 role codes
│   │   ├── ContractPerformance.java            ← PF · DL · DQ · DF · MA
· TE
```

```
│   │   ├── BusinessDayConventionEnum.java
│   │   ├── DayCountConvention.java
│   │   ├── EndOfMonthConventionEnum.java
│   │   ├── Calendar.java
│   │   ├── FeeBasis.java
│   │   ├── ScalingEffect.java
│   │   ├── PenaltyType.java
│   │   ├── InterestCalculationBase.java
│   │   ├── OptionType.java
│   │   ├── OptionExerciseType.java
│   │   ├── DeliverySettlement.java
│   │   ├── ClearingHouse.java
│   │   ├── Seniority.java
│   │   ├── GuaranteedExposure.java
│   │   ├── CreditEventTypeCovered.java
│   │   ├── PrepaymentEffect.java
│   │   ├── CyclePointOfInterestPayment.java
│   │   ├── CyclePointOfRateReset.java
│   │   ├── ReferenceRole.java
│   │   ├── ReferenceType.java
│   │   └── ContractReference.java              ← class (not enum)
│   │
│   └── util/
│       ├── CommonUtils.java                    ← isNull(), FX rate
helper
│       ├── CycleUtils.java                     ← cycle string parsing
│       ├── RedemptionUtils.java                ← principal redemption
helpers
│       ├── StringUtils.java                    ← convention string
constants
│       └── Constants.java                      ← MAX_LIFETIME (50 years)
│
└── test/java/org/actus/
    ├── contracts/  (19 test classes + AD0Test)
    ├── conventions/  (10 test classes)
    ├── time/  (3 test classes)
    ├── util/  (CycleUtilsTest)
    ├── attributes/ (ContractModelTest)
    └── testutils/
        ├── ContractTestUtils.java
        ├── TestData.java
        ├── DataObserver.java               ← implements
RiskFactorModelProvider
        ├── ObservedDataSet.java
        ├── ObservedDataPoint.java
        ├── ObservedEvent.java
        └── ResultSet.java
```

Package Dependency Map

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    subgraph "Public API"
        CT["org.actus.contracts\nContractType (dispatcher)"]
        ATTR["org.actus.attributes\nContractModelProvider"]
        EXT["org.actus.externals\nRiskFactorModelProvider"]
    end

    subgraph "Contract Layer"
        IMPL["org.actus.contracts\n19 CT implementations"]
    end

    subgraph "Event Layer"
        EV["org.actus.events\nContractEvent · EventFactory · EventSequence"]
    end

    subgraph "Function Layer"
        FN["org.actus.functions\nPayOffFunction · StateTransitionFunction\n200+
    end

    subgraph "State Layer"
        SS["org.actus.states\nStateSpace"]
    end

    subgraph "Convention Layer"
        CONV["org.actus.conventions\nbusinessday · daycount · endofmonth · cont
    end

    subgraph "Time Layer"
        TIME["org.actus.time\nScheduleFactory · CycleAdjuster · Calendars"]
    end

    subgraph "Type System"
        TYPES["org.actus.types\n30+ enums"]
    end

    subgraph "Utilities"
        UTIL["org.actus.util\nCommonUtils · CycleUtils · Constants"]
    end

    CT --> IMPL
    IMPL --> EV
    IMPL --> TIME
    EV --> FN
    FN --> SS
    FN --> ATTR
    FN --> EXT
```

```
    FN --> CONV
    EV --> CONV
    TIME --> CONV
    IMPL --> CONV
    IMPL --> TYPES
    IMPL --> UTIL
    FN --> UTIL
    ATTR --> TYPES
    EV --> TYPES
    SS --> TYPES
```

Class Count by Package

Package	Interfaces	Classes	Enums	Total
`org.actus`	0	3 (exceptions)	0	3
`org.actus.attributes`	1	1	0	2
`org.actus.contracts`	0	20	0	20
`org.actus.events`	0	3	0	3
`org.actus.externals`	1	0	0	1
`org.actus.functions`	2	200+	0	200+
`org.actus.states`	0	1	0	1
`org.actus.conventions`	3	12	0	15
`org.actus.time`	2	7	0	9
`org.actus.types`	0	1	29	30
`org.actus.util`	0	5	0	5
Total	**9**	**253+**	**29**	**291+**

Formal Framework

Formal Framework

The ACTUS Technical Specification formally defines each financial contract as a persistable state machine. This page covers the four-component model, the two input types, the output structure, and what the specification defines per contract type. For the motivation behind separating knowns from unknowns, see Fundamentals.

State Machine Abstraction

Each financial contract in ACTUS is defined as a state machine with four components:

- **States (S)** — the contract state vector, holding the current values of all state variables
- **Events (E)** — discrete points in time at which cash flows are exchanged or states are updated
- **Transitions (T)** — the state transition functions that map a pre-event state to a post-event state
- **Payoffs (P)** — the payoff functions that compute the cash flow at each event

When a contract is evaluated at an event `e`, a counterparty receives a payout `P(e)`, which can be positive or negative.

Inputs

An ACTUS Contract Type requires two types of input:

Contract Terms — the legal parameters defining the contract, drawn from the ACTUS Data Dictionary. Contract terms include elements such as the notional principal, interest rate, maturity date, and applicable cycles and conventions.

Risk Factor States — the current and future states of market, counterparty, and behavioral risk factors such as interest rates, exchange rates, and credit indicators. Risk factors are held in databases outside the contract definition itself.

Outputs

The output of an ACTUS Contract Type is a set of **Contract Events (CE)**. Contract Events denote real-world transaction events such as the initiation of a contract deal, the maturity of

a contract, or interest and dividend payments. Each contract event has a time, an event type, a payoff amount, and a resulting state.

Contract State

The contract state is a vector of state variables that changes as each event is processed. State variables track quantities such as the current notional principal, accrued interest, the nominal interest rate, the performance status of the contract, and scaling multipliers.

Each event is processed at its scheduled time: the state transition function (STF) updates the state variables, and the payoff function (POF) computes the resulting cash flow. Both functions are defined in STF and POF Functions.

Event Sequencing

When contract events of different types occur at the same time, the sequence in which their STF and POF are evaluated is decisive for the resulting cash flows and state transitions. An event sequence indicator is used to define the processing order of events that occur at the exact same timestamp.

Three-Part Contract Definition

For each contract type, the specification defines three things:

1. **Schedule generation** — the algorithm that produces the ordered list of contract events from the contract terms (see Schedule Generation)
2. **State initialization** — the initial values of all state variables, derived from the contract terms at the status date
3. **STF and POF functions** — one per applicable event type (see STF and POF Functions)

Public API

Public API

The public API of the ACTUS Java implementation consists of three classes/interfaces. A caller needs nothing else.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ContractType {
        +schedule(LocalDateTime to, ContractModelProvider model) ArrayList~Contr
        +apply(ArrayList~ContractEvent~ events, ContractModelProvider model, Ris
    }
    class ContractModelProvider {
        <<interface>>
        +getAs(String name) T
    }
    class RiskFactorModelProvider {
        <<interface>>
        +keys() Set~String~
        +stateAt(String id, LocalDateTime time, StateSpace states, ContractMode
        +events(ContractModelProvider attributes) Set~ContractEvent~
    }
    class ContractModel {
        -Map~String,Object~ attributes
        +of(Map~String,Object~ attributes) ContractModel
        +parse(Map~String,String~ attributes) ContractModel
        +getAs(String name) T
    }
    ContractModelProvider <|.. ContractModel
    ContractType ..> ContractModelProvider : reads
    ContractType ..> RiskFactorModelProvider : reads
```

ContractType

`org.actus.contracts.ContractType` — `public final class`

The single dispatcher. Both methods inspect the `contractType` attribute from the model and route to the corresponding implementation class.

`schedule`

```
public static ArrayList<ContractEvent> schedule(
    LocalDateTime to,
    ContractModelProvider model)
```

Generates all contract events from the contract's status date up to (and including) `to`. The returned list is sorted by event time using `EventSequence` offsets, so simultaneous events of different types are in the correct processing order.

`to` is typically set to the analysis horizon — the furthest point in time for which you want projected cash flows.

`apply`

```
public static ArrayList<ContractEvent> apply(
    ArrayList<ContractEvent> events,
    ContractModelProvider model,
    RiskFactorModelProvider observer)
```

Evaluates each event in sequence. For each event, the payoff function computes the cash flow and the state transition function updates the `StateSpace`. The returned list is the same as `events` but each `ContractEvent` now has a populated `payoff` value and a post-event `StateSpace` snapshot.

Dispatch Table

`contractType` attribute	Dispatched class
`PAM`	`PrincipalAtMaturity`
`LAM`	`LinearAmortizer`
`NAM`	`NegativeAmortizer`
`ANN`	`Annuity`
`LAX`	`ExoticLinearAmortizer`
`CLM`	`CallMoney`
`UMP`	`UndefinedMaturityProfile`
`CSH`	`Cash`

`contractType` attribute	Dispatched class
`STK`	`Stock`
`COM`	`Commodity`
`FXOUT`	`ForeignExchangeOutright`
`SWPPV`	`PlainVanillaInterestRateSwap`
`SWAPS`	`Swap`
`CAPFL`	`CapFloor`
`OPTNS`	`Option`
`FUTUR`	`Future`
`CEG`	`CreditEnhancementGuarantee`
`CEC`	`CreditEnhancementCollateral`
`BCS`	`BoundaryControlledSwitch`

Unknown contract type → throws `ContractTypeUnknownException`.

ContractModelProvider

`org.actus.attributes.ContractModelProvider` — `public interface`

```
public interface ContractModelProvider {
    <T> T getAs(String name);
}
```

All contract terms are retrieved through this single generic method. The type parameter `T` is inferred from the call site. Attribute names match the ACTUS data dictionary exactly (e.g. `"notionalPrincipal"`, `"nominalInterestRate"`, `"maturityDate"`).

Accessing a missing or unconvertible attribute throws `AttributeConversionException`.

ContractModel

The bundled implementation. It wraps a `Map<String, Object>`.

```
// Build from already-typed objects
ContractModel model = ContractModel.of(Map.of(
    "contractType",      ContractTypeEnum.PAM,
    "notionalPrincipal", 1_000_000.0,
    "maturityDate",      LocalDateTime.of(2030, 1, 1, 0, 0)
));

// Build from a map of strings (parses to correct types internally)
ContractModel model = ContractModel.parse(Map.of(
    "contractType",      "PAM",
    "notionalPrincipal", "1000000.0",
    "maturityDate",      "2030-01-01T00:00:00"
));
```

`ContractModel.parse` converts each String value to the correct Java type as defined by the ACTUS data dictionary — enums, doubles, `LocalDateTime`, `Period`, etc. Unrecognised values throw `AttributeConversionException`.

RiskFactorModelProvider

`org.actus.externals.RiskFactorModelProvider` — `public interface`

```
public interface RiskFactorModelProvider {

    // Returns all known risk factor identifiers
    Set<String> keys();

    // Returns the state of risk factor 'id' at 'time'
    double stateAt(String id, LocalDateTime time,
                   StateSpace states,
                   ContractModelProvider attributes,
                   boolean isMarket);

    // Returns any contingent events injected by the risk model (default:
empty set)
    default Set<ContractEvent> events(ContractModelProvider attributes) {
        return new HashSet<>();
    }
}
```

The implementor is fully responsible for market data. ACTUS never stores or owns risk factor values — it only calls `stateAt` at evaluation time. This makes it trivial to plug in scenario generators, historical data sources, or simulation engines.

Risk factors are identified by string IDs that match the `marketObjectCode` attribute in the contract model (e.g. a rate index like `"USD-LIBOR-3M"` or an FX pair like `"USD/EUR"`).

If `stateAt` is called for an unknown ID, the implementation should throw `RiskFactorNotFoundException`.

The `isMarket` flag distinguishes market rate factors (`true`) from behavioural or credit factors (`false`).

Typical Implementation Pattern

```
public class ScenarioObserver implements RiskFactorModelProvider {
    private final Map<String, NavigableMap<LocalDateTime, Double>>
timeSeries;

    @Override
    public Set<String> keys() { return timeSeries.keySet(); }

    @Override
    public double stateAt(String id, LocalDateTime time, StateSpace
states,
                          ContractModelProvider attrs, boolean isMarket)
{
        NavigableMap<LocalDateTime, Double> ts = timeSeries.get(id);
        if (ts == null) throw new RiskFactorNotFoundException(id);
        // Last Observation Carried Forward (LOCF)
        Map.Entry<LocalDateTime, Double> entry = ts.floorEntry(time);
        return entry != null ? entry.getValue() : 0.0;
    }
}
```

Exceptions

Class	Extends	When Thrown
`AttributeConversionException`	`RuntimeException`	`ContractModel.getAs()` cannot convert the stored value to the requested type

Class	Extends	When Thrown
`ContractTypeUnknownException`	`RuntimeException`	`ContractType` dispatch receives an unrecognised `contractType` string
`RiskFactorNotFoundException`	`RuntimeException`	`RiskFactorModelProvider.stateAt()` is called for an ID not in `keys()`

All three are unchecked exceptions — callers are not required to catch them.

State Transition Functions and Payoff Functions

State Transition Functions and Payoff Functions

Two Function Families

The ACTUS Technical Specification defines two families of functions for each contract type:

- **State Transition Functions (STF)** — update the contract state when an event occurs
- **Payoff Functions (POF)** — compute the cash flow produced by an event

Together, STF and POF fully define the behavior of a contract type at every event.

State Transition Functions (STF)

State Transition Functions define the transition of states from a pre-event to a post-event state when a certain event applies. The pre-event and post-event times are indexed with $\mathbf{t}^-$ and $\mathbf{t}^+$, respectively.

An STF takes as input the contract state at t^- (immediately before the event), the contract terms, and any applicable risk factor values at the event time. It produces as output the updated contract state at t^+ (immediately after the event).

STFs are named according to the pattern: `STF_[EventType]_[ContractType]`

where `[EventType]` refers to the event type code and `[ContractType]` refers to the contract type code to which the STF applies.

Payoff Functions (POF)

Payoff Functions define how the cash flow for a certain event is derived from the current states and from the contract terms. If necessary, the resulting cash flow can be indexed with the event time.

POFs are named according to the pattern: `POF_[EventType]_[ContractType]`

A POF takes as input the contract state at t^- and the contract terms. It returns the cash flow amount — positive if it represents money received by the long party, negative if money is paid.

Event Types

Contract events mark specific points in time during the lifetime of a contract at which a cash flow is being exchanged or the states of the contract are being updated. The specification defines a set of event types, each with a corresponding STF and POF per contract type.

Event types defined in the ACTUS specification include:

Code	Event Type
AD	Monitoring / Analysis Date
IED	Initial Exchange Date
FP	Fee Payment
PR	Principal Redemption
PI	Principal Increase
PRF	Principal Payment Cash Flow
IP	Interest Payment
IPCI	Interest Capitalization
IPCB	Interest Calculation Base Fixing
RR	Rate Reset (fixing)
RRF	Rate Reset (floating)
SC	Scaling Index Fixing
PRD	Purchase Date
TD	Termination Date
MD	Maturity Date
PP	Penalty Payment
STD	Settlement Date
XD	Execution Date
DV	Dividend Payment

Code	Event Type
MR	Margin Call
CD	Credit Default

Event Sequence

When events of different types occur at the same time, the specification assigns each event type an event sequence indicator (priority). This indicator defines the order in which events at the same timestamp are evaluated, which is decisive for the resulting cash flows and state transitions.

Function Naming Convention

The specification uses a consistent naming pattern so that every function is unambiguously identified. For example:

- `STF_IP_PAM` — the State Transition Function for an Interest Payment event on a PAM (Principal at Maturity) contract
- `POF_IP_PAM` — the Payoff Function for an Interest Payment event on a PAM contract
- `STF_RR_ANN` — the State Transition Function for a Rate Reset event on an ANN (Annuity) contract

This pattern scales across all event types and contract types. Where an event type does not apply to a contract type, the STF and POF are defined as no-op (identity) functions.

Relationship to Risk Factors

At certain events — notably Rate Reset (RR, RRF) and Scaling (SC) — the STF requires a current value from an external risk factor (such as an interest rate index or a scaling index). The specification defines the interface through which risk factor values are accessed, but the actual values come from an external risk factor database at evaluation time.

Contract Type Implementations

Contract Type Implementations

Common Pattern

Every contract type class is `public final` with two `static` methods and no state of its own.

```
public final class PrincipalAtMaturity {

    public static ArrayList<ContractEvent> schedule(
            LocalDateTime to, ContractModelProvider model) { ... }

    public static ArrayList<ContractEvent> apply(
            ArrayList<ContractEvent> events,
            ContractModelProvider model,
            RiskFactorModelProvider observer) { ... }
}
```

`schedule` reads contract terms, builds date sets using `ScheduleFactory`, wraps each date into a `ContractEvent` via `EventFactory`, and returns them sorted.

`apply` initialises a `StateSpace` from the model, then iterates over the sorted events calling `event.eval()` on each, accumulating state changes.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
flowchart LR
    A["schedule(to, model)"] --> B["Read contract terms\nfrom model"]
    B --> C["ScheduleFactory\nfor each cycle"]
    C --> D["EventFactory\ncreateEvent / createEvents"]
    D --> E["Sort by epoch offset"]
    E --> F["return ArrayList&lt;ContractEvent&gt;"]

    G["apply(events, model, observer)"] --> H["Initialise StateSpace\nfrom mode
    H --> I["for each ContractEvent"]
    I --> J["event.eval(states, model,\nobserver, dayCounter, bda)"]
    J --> K["POF → payoff double"]
    J --> L["STF → updated StateSpace"]
```

```
    L --> I
    K --> M["return events with\npayoffs + states"]
```

Contract Implementations

PAM — Principal at Maturity

File: `PrincipalAtMaturity.java` (279 lines)

A bullet bond: interest is paid periodically over the life of the contract; the full principal is repaid at maturity.

Events generated:

Event	Condition	Description
`IED`	Always	Initial exchange — principal disbursement
`IP`	`cycleOfInterestPayment` set and not fully capitalised	Periodic interest payments
`IPCI`	`cycleOfInterestPayment` set and capitalisation enabled	Capitalised interest (added to principal)
`IPCB`	`interestCalculationBase` = `NTL`	Interest calculation base fixing
`RR`	`cycleOfRateReset` set, floating	Rate reset — market rate observed
`RRF`	`cycleOfRateReset` set, fixed next rate	Rate reset — next rate fixed in contract
`SC`	`cycleOfScalingIndex` set	Scaling index update
`FP`	`cycleOfFee` set	Fee payment
`PRD`	`purchaseDate` set	Purchase event
`TD`	`terminationDate` set	Early termination
`MD`	Always	Maturity — principal redemption
`AD`	Analysis dates requested	Monitoring / valuation snapshots

State initialisation (representative subset):

```
StateSpace states = new StateSpace();
states.statusDate                = model.getAs("statusDate");
states.notionalPrincipal         = model.getAs("notionalPrincipal");
states.nominalInterestRate       = model.getAs("nominalInterestRate");
states.accruedInterest           = model.getAs("accruedInterest");
states.contractPerformance       = model.getAs("contractPerformance");
states.feeAccrued                = model.getAs("feeAccrued");
states.notionalScalingMultiplier = 1.0;  // or from model
states.interestScalingMultiplier = 1.0;
```

LAM — Linear Amortizer

File: `LinearAmortizer.java` (414 lines)

Fixed periodic principal redemptions. Each `PR` event repays a constant amount of principal, and the interest payment decreases over time as the outstanding principal shrinks.

Adds the `PR` (Principal Redemption) event set to the PAM event set. The `nextPrincipalRedemptionPayment` state variable tracks the next redemption amount.

NAM — Negative Amortizer

File: `NegativeAmortizer.java` (448 lines)

Payments are fixed but may be less than the accrued interest. Any shortfall is capitalised — added back to the outstanding principal — so the principal can *increase* over time before eventually amortising.

ANN — Annuity

File: `Annuity.java` (500 lines)

Level periodic payments. Each payment covers the accrued interest plus a principal redemption amount. The redemption amount grows over time as the outstanding interest shrinks. The `STF_PRF_ANN` function recomputes the `nextPrincipalRedemptionPayment` state variable at each `PRF` (Payment Recalculation) event.

LAX — Exotic Linear Amortizer

File: `ExoticLinearAmortizer.java` (523 lines)

A linear amortiser with additional structural features — variable redemption schedules, exotic rate reset patterns, or complex fee structures. The most complex of the amortising instruments in terms of schedule generation.

CLM — Call Money

File: `CallMoney.java`

Short-term credit facility. Either party may call the outstanding balance at short notice. The maturity is not fixed at inception; it is determined by an exercise event.

UMP — Undefined Maturity Profile

File: `UndefinedMaturityProfile.java`

Open-ended instrument where the balance evolves according to market risk factors rather than a fixed amortisation schedule. Maximum lifetime capped at `Constants.MAX_LIFETIME_UMP` (10 years).

CSH — Cash

File: `Cash.java`

A current account or cash holding. No interest; the principal is constant. Used to model unencumbered cash positions.

STK — Stock

File: `Stock.java`

Equity instrument. The schedule includes `DV` (Dividend) events based on the dividend cycle. The stock price is retrieved from the `RiskFactorModelProvider` at each evaluation date.

Maximum schedule horizon: `Constants.MAX_LIFETIME_STK` (10 years).

COM — Commodity

File: `Commodity.java`

Physical or financial commodity. Similar to STK but with commodity-specific payoff conventions and `DeliverySettlement` logic.

FXOUT — Foreign Exchange Outright

File: `ForeignExchangeOutright.java`

A spot or forward exchange of two currency amounts. Generates two legs with `STD` (Settlement Date) events. The payoff is computed using the FX rate from the risk factor model.

SWPPV — Plain Vanilla Interest Rate Swap

File: `PlainVanillaInterestRateSwap.java`

Fixed-for-floating interest rate swap. Generates two internal legs — a PAM-like fixed leg and a floating-rate leg. Both legs share a single `StateSpace`. The `ContractReference` mechanism links the two legs.

SWAPS — Swap

File: `Swap.java`

Generic swap container. Delegates to two referenced sub-contracts (Leg 1 and Leg 2), each of which can be any contract type.

CAPFL — Cap/Floor

File: `CapFloor.java`

Interest rate cap or floor. At each caplet/floorlet date, the payoff is `max(rate − strike, 0) × notional × dayFraction` (cap) or `max(strike − rate, 0) × notional × dayFraction` (floor). Rates are observed from the risk factor model.

OPTNS — Option

File: `Option.java`

European or American option on an underlying contract. The `OptionType` (call or put), `OptionExerciseType` (European, American, Bermudan), and `DeliverySettlement` attributes control behaviour. The `exerciseDate` and `exerciseAmount` state variables track option exercise.

FUTUR — Future

File: `Future.java`

Exchange-traded futures contract. Daily margin calls (`MR` events) are generated based on the `ClearingHouse` mark-to-market. Settlement is at the `STD` event.

CEG — Credit Enhancement Guarantee

File: `CreditEnhancementGuarantee.java`

Creates a triangular relationship: guarantor, obligee, and debtor. The guarantor's obligation mirrors the debtor's obligation — if the debtor defaults, the guarantee is called. The `GuaranteedExposure` attribute controls the coverage fraction.

Uses `ContractReference` to link to the underlying (covered) contract.

CEC — Credit Enhancement Collateral

File: `CreditEnhancementCollateral.java`

Similar to CEG but coverage comes from a pledged collateral asset rather than a guarantor's personal obligation. The collateral value is retrieved from the risk factor model.

BCS — Boundary Controlled Switch

File: `BoundaryControlledSwitch.java`

A meta-contract that monitors boundary conditions and switches between two active legs. Uses four state flags: `boundaryCrossedFlag`, `boundaryMonitoringFlag`, `boundaryLeg1ActiveFlag`, `boundaryLeg2ActiveFlag`.

When the monitored risk factor crosses the defined boundary, a `ME` (Monitoring Event) triggers a switch between legs.

Class Relationships

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ContractType {
        +schedule()
        +apply()
    }
    class PrincipalAtMaturity { +schedule() +apply() }
    class LinearAmortizer { +schedule() +apply() }
    class NegativeAmortizer { +schedule() +apply() }
    class Annuity { +schedule() +apply() }
    class ExoticLinearAmortizer { +schedule() +apply() }
    class Swap { +schedule() +apply() }
    class PlainVanillaInterestRateSwap { +schedule() +apply() }
    class CreditEnhancementGuarantee { +schedule() +apply() }
    class BoundaryControlledSwitch { +schedule() +apply() }
    class OtherContracts["CallMoney, Cash, Stock, Commodity, etc."] { +schedule

    ContractType --> PrincipalAtMaturity : PAM
    ContractType --> LinearAmortizer : LAM
    ContractType --> NegativeAmortizer : NAM
    ContractType --> Annuity : ANN
    ContractType --> ExoticLinearAmortizer : LAX
    ContractType --> Swap : SWAPS
    ContractType --> PlainVanillaInterestRateSwap : SWPPV
    ContractType --> CreditEnhancementGuarantee : CEG
    ContractType --> BoundaryControlledSwitch : BCS
    ContractType --> OtherContracts : others
```

Approximate Implementation Size

Contract	File	Lines
PAM	`PrincipalAtMaturity.java`	279
LAM	`LinearAmortizer.java`	414
NAM	`NegativeAmortizer.java`	448

Contract	File	Lines
ANN	`Annuity.java`	500
LAX	`ExoticLinearAmortizer.java`	523
Others	each ~200–400	varies

Schedule Generation

Schedule Generation

Purpose

The ACTUS Technical Specification defines a deterministic schedule generation algorithm for each contract type. Given a set of contract terms, the algorithm produces an ordered sequence of contract events — each with a timestamp and event type — that represents all contractual obligations over the lifetime of the contract.

Schedule as the Core Output

The schedule is the primary artifact from which cash flows and state transitions are derived. An engine implementing the specification processes events in schedule order: for each event, it calls the payoff function (POF) to compute the cash flow and the state transition function (STF) to update the contract state.

Cycle Notation

Contract terms include cycle definitions that specify the frequency and behavior of recurring events. The specification defines a standard cycle notation:

`P[n][unit]L[stub]`

Where:

- `P` — period prefix (ISO 8601 duration convention)
- `[n]` — the number of units per cycle period
- `[unit]` — the time unit: `D` (days), `W` (weeks), `M` (months), `Q` (quarters), `H` (half-years), `Y` (years)
- `L` — stub indicator prefix
- `[stub]` — stub type: `0` for a long stub at the start or end, `1` for a short stub

For example, `P3ML0` means a quarterly cycle (3 months) with a long stub.

Schedule Construction

For each event type that has a cycle defined in the contract terms, the schedule generation algorithm:

1. Takes the cycle anchor date (start of the schedule)
2. Applies the cycle period repeatedly up to the maturity or termination date
3. Adjusts each generated date according to the applicable end-of-month convention
4. Applies business day adjustment using the specified business day convention and calendar
5. Handles stub periods at the beginning or end of the schedule

Business Day and Calendar Conventions

The specification defines conventions for adjusting dates that fall on non-business days:

- The **Business Day Convention** specifies how to shift a scheduled date to the nearest business day: following, modified following, preceding, modified preceding, and others.
- The **Calendar** identifies which days are business days.
- The **End-of-Month Convention** governs whether cycles anchored at the end of a month continue to generate end-of-month dates.
- The **Calc/Shift** convention determines whether accrual periods are calculated before or after date adjustment.

Day Count Conventions

Interest accrual between events depends on the elapsed time measured according to a day count convention. The specification defines several day count conventions that are standard in the financial industry, specifying how the number of days and the year basis are calculated between two dates.

Event Priority

When multiple event types generate events at the same date, the specification defines an event sequence indicator — a priority order that determines which event is processed first. This is critical because the state at t^- passed to a later event depends on all earlier events at the same timestamp having already been processed.

Contract Events

Contract Events

Overview

A `ContractEvent` is the fundamental unit of computation. It carries everything needed to evaluate a single point in the contract's lifecycle: its timestamp, its type, references to the payoff and state-transition functions to call, and — after evaluation — its computed payoff and post-event state.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ContractEvent {
        -long epochOffset
        -LocalDateTime scheduleTime
        -LocalDateTime eventTime
        -EventType eventType
        -String currency
        -double payoff
        -String contractID
        -StateSpace states
        -PayOffFunction fPayOff
        -StateTransitionFunction fStateTrans
        +eval(StateSpace, ContractModelProvider, RiskFactorModelProvider, DayCou
        +getContractID() String
        +compareTo(ContractEvent) int
    }
    class EventFactory {
        +createEvent(scheduleTime, type, currency, pof, stf, contractID) Contrac
        +createEvent(scheduleTime, type, currency, pof, stf, bda, contractID) Co
        +createEvents(Set~LocalDateTime~, type, currency, pof, stf, contractID)
        +createEvents(Set~LocalDateTime~, type, currency, pof, stf, bda, contrac
    }
    class EventSequence {
        +upperBound() int
        +timeOffset(EventType) int
    }
    EventFactory --> ContractEvent : creates
    ContractEvent --> EventSequence : uses for epochOffset
```

ContractEvent

`org.actus.events.ContractEvent` — `public final class implements Comparable<ContractEvent>` (256 lines)

Fields

Field	Type	Description
`epochOffset`	`long`	Sort key: epoch milliseconds of event time + event-type sequence offset
`scheduleTime`	`LocalDateTime`	The contractually scheduled date (used for interest accrual calculations)
`eventTime`	`LocalDateTime`	The actual settlement date after business day adjustment
`eventType`	`EventType`	The event type code (IP, MD, RR, etc.)
`currency`	`String`	The currency of the cash flow
`payoff`	`double`	Computed payoff amount — positive = received, negative = paid
`states`	`StateSpace`	The post-event contract state vector
`contractID`	`String`	The originating contract identifier
`fPayOff`	`PayOffFunction`	Lambda/class reference assigned at creation; called during eval()
`fStateTrans`	`StateTransitionFunction`	Lambda/class reference assigned at creation; called during eval()

Two Times: `scheduleTime` vs `eventTime`

This design is critical. The ACTUS specification distinguishes *when a payment is contractually scheduled* (the schedule time) from *when it actually settles* (the event time, after business day adjustment). Many payoff calculations — particularly interest accrual — must use the schedule time, not the settlement time.

```
scheduleTime = "2025-03-31"  (contractual — last day of March)
eventTime    = "2025-04-01"  (actual — first business day of April,
because March 31 is a Sunday)
```

```
Interest accrual period → uses scheduleTime (March 31)
Bank transfer           → happens on eventTime (April 1)
```

`eval` Method

```
public void eval(StateSpace states,
                 ContractModelProvider model,
                 RiskFactorModelProvider observer,
                 DayCountCalculator dayCounter,
                 BusinessDayAdjuster timeAdjuster)
```

1. Calls `fPayOff.eval(eventTime, states, model, observer, dayCounter, timeAdjuster)` → stores result in `this.payoff`
2. Calls `fStateTrans.eval(eventTime, states, model, observer, dayCounter, timeAdjuster)` → stores result in `this.states`

Both functions receive the same pre-event `states` object. The STF returns a *new* (or mutated) StateSpace that becomes the input for the next event.

`compareTo`

Events are ordered by `epochOffset`, which is computed as:

```
epochOffset = eventTime.toEpochMilli() +
EventSequence.timeOffset(eventType)
```

This ensures:

- Events at different times are sorted chronologically
- Events at the *same* time are sorted by event type, in the sequence defined by the ACTUS specification

---

EventFactory

`org.actus.events.EventFactory` — `public final class` (114 lines)

A utility class for assembling `ContractEvent` objects. All methods are static.

Single-event creation

```
// Without business day adjustment (eventTime == scheduleTime)
ContractEvent e = EventFactory.createEvent(
    scheduleTime,   // LocalDateTime
    eventType,      // EventType
    currency,       // String
    payOff,         // PayOffFunction
    stateTrans,     // StateTransitionFunction
    contractID      // String
);

// With business day adjustment (eventTime =
bda.shiftEventTime(scheduleTime))
ContractEvent e = EventFactory.createEvent(
    scheduleTime, eventType, currency,
    payOff, stateTrans,
    bda,          // BusinessDayAdjuster
    contractID
);
```

Batch creation from a schedule set

```
// Creates one ContractEvent per date in eventSchedule, all with the same
type/functions
Set<ContractEvent> events = EventFactory.createEvents(
    eventSchedule,  // Set<LocalDateTime> from ScheduleFactory
    eventType, currency, payOff, stateTrans, contractID
);
```

The batch variant is the most common pattern inside contract type implementations:

```
// Typical usage inside PrincipalAtMaturity.schedule()
Set<LocalDateTime> ipDates = ScheduleFactory.createSchedule(
    ipAnchor, maturityDate, cycleOfIP, eomConvention);

events.addAll(EventFactory.createEvents(
    ipDates, EventType.IP, currency,
    new POF_IP_PAM(), new STF_IP_PAM(),
    businessDayAdjuster, contractID));
```

EventSequence

`org.actus.events.EventSequence` — `public final class` (115 lines)

Defines the integer time offset added to each event's epoch millisecond to establish the processing order when multiple events fall on the same timestamp. A lower offset means earlier processing.

Time Offsets (selected)

Event Type	Offset	Reason for order
`AD`	10	Analysis date — snapshot before anything else
`IED`	20	Initial exchange — must precede any accrual
`IPCB`	24	Base fixing — before interest payment
`IP`	40	Interest — after principal events at same date
`IPCI`	40	Capitalisation — same priority as IP
`FP`	60	Fees — after core cash flows
`DV`	70	Dividend — after fees
`PR`	30	Principal redemption — before interest at month end
`RR`	100	Rate reset — after all payments at the reset date
`RRF`	100	Fixed rate reset — same as RR
`SC`	100	Scaling — after payments, before next period begins
`TD`	varies	Termination
`MD`	varies	Maturity
`CD`	500	Credit default — last in sequence

`EventSequence.upperBound()` returns `900` — the maximum possible offset, used as a guard for event filtering.

Why This Matters

Consider a contract where principal redemption (PR) and interest payment (IP) both fall on June 30. The interest for the period must be computed on the *outstanding principal before*

redemption, so PR must be processed first. The offsets enforce this automatically — no contract implementation needs to sort manually.

Event Lifecycle Summary

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
stateDiagram-v2
    [*] --> Created : EventFactory.createEvent()
    Created --> Sorted : Added to ArrayList, sorted by epochOffset
    Sorted --> Evaluated : ContractEvent.eval() called in apply()
    Evaluated --> [*] : payoff and states populated
```

Phase	What happens
Creation	`scheduleTime` and functions (`fPayOff`, `fStateTrans`) set; `payoff = 0`, `states = null`
Sorting	`compareTo` orders the list by `epochOffset` before `apply()` begins
Evaluation	`eval()` calls both functions; `payoff` and `states` are filled in
Result	Caller reads `getPayoff()` and `getStates()` from each event

Contract Types in the Specification

Contract Types in the Specification

Overview

The ACTUS Technical Specification formally defines each contract type as a state machine: it specifies the schedule generation algorithm, the state initialization logic, and the STF/POF pair for every event type applicable to that contract.

ACTUS represents virtually all financial instruments by parameterizing and combining a set of contract types, each capturing a distinct cash flow pattern. The specification covers approximately 32 contract types across several categories.

Fixed Income — With Maturities

These contract types model instruments that have a defined end date (maturity).

Code	Name	Cash Flow Pattern
PAM	Principal at Maturity	Periodic interest payments; full principal repaid at maturity
ANN	Annuity	Level periodic payments covering both interest and principal
NAM	Negative Amortizer	Payments may be less than accrued interest; shortfall capitalizes
LAM	Linear Amortizer	Fixed periodic principal redemptions with declining interest
ANX	Exotic Annuity	Annuity variant with additional structural features
NAX	Exotic Negative Amortizer	Negative amortizer variant with additional structural features
LAX	Exotic Linear Amortizer	Linear amortizer variant with additional structural features
CLM	Call Money	Short-term credit facility callable by either party
PBN	Perpetual Bond	Coupon-only; no maturity; principal not redeemed

Fixed Income — Without Maturities

These contract types model open-ended or demand instruments.

Code	Name	Description
CSH	Cash	Current account or cash holding
UMP	Undefined Maturity Profile	Instrument with no fixed maturity and market-risk-driven balance

Index-Based

Code	Name	Description
STK	Stock	Equity instrument with dividend payments
COM	Commodity	Physical or financial commodity instrument

Combined Symmetric (Derivatives)

Symmetric instruments where both counterparties have obligations.

Code	Name	Description
SWAPS	Swap	Exchange of cash flows between two legs
SWPPV	Plain Vanilla Swap	Fixed-for-floating interest rate swap
FXOUT	Foreign Exchange Outright	Spot or forward exchange of two currencies
FUTUR	Future	Exchange-traded futures contract

Combined Asymmetric (Options)

Asymmetric instruments where one party has the right but not the obligation.

Code	Name	Description
OPTNS	Option	Right to buy or sell an underlying at a given price
CAPFL	Cap/Floor	Interest rate cap or floor

Code	Name	Description
BNDCP	Bond with Call/Put	Bond with embedded call or put option
BNDWR	Bond with Warrant	Bond with attached warrant
EXOTI	Exotic Option	Non-standard option payoff structure

Securitization

Code	Name	Description
CDSWP	Credit Default Swap	Protection on a reference credit
TRSWP	Total Return Swap	Exchange of total return of an asset for a fixed or floating rate
CLNTE	Credit Linked Note	Debt instrument with embedded credit derivative
SCRMR	Securitization — Mortgage	Mortgage-backed securitization structure
SCRCR	Securitization — Corporate	Corporate-backed securitization structure

Credit Enhancement

Code	Name	Description
CEG	Guarantee	Creates a relationship between a guarantor, an obligee, and a debtor, moving the exposure from the debtor to the guarantor
CEC	Collateral	Creates a relationship between a collateral, an obligee, and a debtor, covering the exposure from the debtor with the collateral
BCS	Basic Credit Sheet	Simplified credit support instrument

What the Specification Defines Per Contract Type

For every contract type, the ACTUS Technical Specification provides:

1. **Applicable contract terms** — the subset of data dictionary attributes that are mandatory, optional, or non-applicable for this contract type, following the attribute applicability rules
2. **Schedule generation algorithm** — a step-by-step description of how to generate the event schedule from the contract terms, including which cycles generate which event types
3. **State initialization** — the initial values of all state variables, derived from the contract terms at the status date
4. **State Transition Functions** — one STF per applicable event type, defining how the state variables change when that event is processed
5. **Payoff Functions** — one POF per applicable event type, defining how the cash flow is computed from the state variables and contract terms

The specification is written for all contract types in a consistent format, enabling engine implementations to follow a uniform processing model regardless of contract type.

Payoff and State Transition Functions

Payoff and State Transition Functions

The Two Interfaces

Every computation in the ACTUS engine reduces to calling one of two interfaces:

```
// Computes the cash flow for an event
public interface PayOffFunction {
    double eval(
        LocalDateTime time,
        StateSpace states,
        ContractModelProvider model,
        RiskFactorModelProvider riskFactorModel,
        DayCountCalculator dayCounter,
        BusinessDayAdjuster timeAdjuster
    );
}

// Updates the contract state after an event
public interface StateTransitionFunction {
    StateSpace eval(
        LocalDateTime time,
        StateSpace states,
        ContractModelProvider model,
        RiskFactorModelProvider riskFactorModel,
        DayCountCalculator dayCounter,
        BusinessDayAdjuster timeAdjuster
    );
}
```

Both receive the same six arguments — the event time, the pre-event state, contract terms, market data, and the two convention calculators. `PayOffFunction` returns a `double` (the cash flow). `StateTransitionFunction` returns a `StateSpace` (the post-event state).

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class PayOffFunction {
```

```
        <<interface>>
        +eval(time, states, model, riskFactorModel, dayCounter, timeAdjuster) d
    }
    class StateTransitionFunction {
        <<interface>>
        +eval(time, states, model, riskFactorModel, dayCounter, timeAdjuster) S
    }
    class POF_IED_PAM { +eval(...) double }
    class POF_IP_PAM  { +eval(...) double }
    class POF_MD_PAM  { +eval(...) double }
    class STF_IED_PAM { +eval(...) StateSpace }
    class STF_IP_PAM  { +eval(...) StateSpace }
    class STF_MD_PAM  { +eval(...) StateSpace }

    PayOffFunction <|.. POF_IED_PAM
    PayOffFunction <|.. POF_IP_PAM
    PayOffFunction <|.. POF_MD_PAM
    StateTransitionFunction <|.. STF_IED_PAM
    StateTransitionFunction <|.. STF_IP_PAM
    StateTransitionFunction <|.. STF_MD_PAM
```

Naming Convention

Every implementation class follows a strict naming pattern:

```
[POF|STF]_[EventType]_[ContractType]
```

Part	Values	Meaning
`POF`	`POF`	Payoff Function
`STF`	`STF`	State Transition Function
`[EventType]`	`IED`, `IP`, `MD`, `RR`, …	The event type this function handles
`[ContractType]`	`PAM`, `LAM`, `ANN`, …	The contract type it applies to

Examples:

- `POF_IED_PAM` — the payoff for the Initial Exchange Date on a PAM contract
- `STF_RR_LAM` — the state transition for a Rate Reset on a Linear Amortizer
- `POF_IP_ANN` — the interest payment payoff for an Annuity

Example Implementations

`POF_IED_PAM` — Initial Exchange, PAM

```
public final class POF_IED_PAM implements PayOffFunction {
    public double eval(LocalDateTime time, StateSpace states,
                       ContractModelProvider model,
                       RiskFactorModelProvider riskFactorModel,
                       DayCountCalculator dayCounter,
                       BusinessDayAdjuster timeAdjuster) {
        return CommonUtils.settlementCurrencyFxRate(riskFactorModel,
model, time, states)
             *
ContractRoleConvention.roleSign(model.getAs("contractRole"))
             * (-1)
             * (model.<Double>getAs("notionalPrincipal")
                + model.<Double>getAs("premiumDiscountAtIED"));
    }
}
```

The initial exchange disburses the notional principal plus any premium or discount. The role sign (`+1` or `-1`) determines the direction from the perspective of the contract's role (lender vs. borrower). The FX rate converts to the settlement currency if needed.

`POF_IP_PAM` — Interest Payment, PAM

```
// Simplified representation of the logic
return fxRate
     * roleSign
     * (states.accruedInterest
        + states.nominalInterestRate
          * dayCounter.dayCountFraction(lastIP, time)
          * states.notionalPrincipal
          * states.interestScalingMultiplier);
```

Interest = accrued interest carried in state + incremental interest for the current period. The day count fraction converts the time interval to a year fraction.

`STF_IP_PAM` — Interest Payment State Transition, PAM

```
// After paying interest, reset accrued interest to 0 and update status
date
```

```
states.accruedInterest = 0.0;
states.feeAccrued      = 0.0;
states.statusDate      = time;
return states;
```

`STF_RR_PAM` — Rate Reset State Transition, PAM

```
// Accrue interest up to the reset date, then apply new rate
states.accruedInterest += oldRate * dcf * notional * scalingMultiplier;
states.nominalInterestRate = riskFactorModel.stateAt(
    model.getAs("marketObjectCodeOfRateReset"), time, states, model,
true);
// Apply rate caps and floors
states.nominalInterestRate = Math.min(
    model.getAs("lifeCap"), states.nominalInterestRate);
states.nominalInterestRate = Math.max(
    model.getAs("lifeFloor"), states.nominalInterestRate);
states.statusDate = time;
return states;
```

Function Inventory by Contract Type

PAM — Principal at Maturity (27 classes)

Function	Type	Event
`POF_AD_PAM`	POF	Analysis Date
`POF_CD_PAM`	POF	Credit Default
`POF_FP_PAM`	POF	Fee Payment
`POF_IED_PAM`	POF	Initial Exchange
`POF_IP_PAM`	POF	Interest Payment
`POF_IPCI_PAM`	POF	Interest Capitalisation
`POF_MD_PAM`	POF	Maturity
`POF_PP_PAM`	POF	Prepayment
`POF_PRD_PAM`	POF	Purchase Date

Function	Type	Event
`POF_PR_PAM`	POF	Principal Redemption
`POF_PY_PAM`	POF	Penalty
`POF_RR_PAM`	POF	Rate Reset (floating)
`POF_RRY_PAM`	POF	Rate Reset (fixed)
`POF_SC_PAM`	POF	Scaling
`POF_TD_PAM`	POF	Termination
`STF_AD_PAM`	STF	Analysis Date
`STF_CD_PAM`	STF	Credit Default
`STF_FP_PAM`	STF	Fee Payment
`STF_IED_PAM`	STF	Initial Exchange
`STF_IP_PAM`	STF	Interest Payment
`STF_IPCI_PAM`	STF	Interest Capitalisation
`STF_MD_PAM`	STF	Maturity
`STF_PP_PAM`	STF	Prepayment
`STF_PRD_PAM`	STF	Purchase Date
`STF_RR_PAM`	STF	Rate Reset (floating)
`STF_RRF_PAM`	STF	Rate Reset (fixed)
`STF_SC_PAM`	STF	Scaling
`STF_TD_PAM`	STF	Termination

LAM — Linear Amortizer (18 classes)

Extends the PAM set with PR (Principal Redemption) functions: `POF_PR_LAM`, `STF_PR_LAM`, `STF_PRF_LAM` (payment recalculation), plus all variants carrying LAM-specific redemption logic.

NAM — Negative Amortizer (3 classes)

`POF_IP_NAM`, `STF_IP_NAM`, `STF_PRF_NAM` — overrides the IP payoff to handle shortfall capitalisation.

LAX — Exotic Linear Amortizer (8 classes)

Full set of overrides for the exotic redemption and interest capitalisation patterns.

ANN — Annuity (1 class)

`STF_PRF_ANN` — recomputes `nextPrincipalRedemptionPayment` so that the combined interest + principal payment remains level.

CLM — Call Money (3 classes)

`POF_PRD_CLM`, `POF_TD_CLM`, `STF_MD_CLM`

CSH — Cash (1 class)

`POF_MD_CSH`

STK — Stock (7 classes)

`POF_DV_STK`, `STF_DV_STK`, and variants for purchase, termination, and monitoring dates.

FXOUT — Foreign Exchange Outright (8 classes)

`POF_STD_FXOUT` (settlement), `STF_STD_FXOUT`, plus purchase, termination, and maturity overrides.

SWPPV — Plain Vanilla Swap (11 classes)

Covers both fixed and floating legs: `POF_IP_SWPPV`, `STF_IP_SWPPV`, `STF_RR_SWPPV`, `STF_RRF_SWPPV`, plus termination and maturity.

SWAPS — Swap (5 classes)

Generic swap delegation: `POF_TD_SWAPS`, `STF_TD_SWAPS`, and leg-switching functions.

CAPFL — Cap/Floor (2 classes)

`POF_IP_CAPFL` — computes `max(rate - strike, 0)` or `max(strike - rate, 0)` times the day fraction and notional. `STF_IP_CAPFL`.

OPTNS — Options (8 classes)

`POF_XD_OPTNS` (execution date payoff), `STF_XD_OPTNS`, `STF_PRD_OPTNS`, `STF_TD_OPTNS`, and variants for European and American exercise.

FUTUR — Futures (4 classes)

`POF_MR_FUTUR` (margin call), `STF_MR_FUTUR`, `POF_STD_FUTUR`, `STF_STD_FUTUR`.

CEG — Credit Enhancement Guarantee (9 classes)

Includes `STF_MD_BCS`, `STF_TD_BCS`, `POF_PRD_BCS`, `POF_TD_BCS` for the guarantee call events and the underlying exposure tracking.

CEC — Credit Enhancement Collateral (3 classes)

`POF_PRD_CEC`, `POF_TD_CEC`, `STF_MD_CEC`.

BCS — Boundary Controlled Switch (4 classes)

`POF_PRD_BCS`, `POF_TD_BCS`, `STF_ME_BCS` (monitoring event — triggers leg switch), `STF_TD_BCS`.

How Functions Are Wired

Contract type implementations wire functions to events at schedule-generation time, not at eval time. This means the function reference is baked into each `ContractEvent` object:

```
// Inside PrincipalAtMaturity.schedule()
events.add(EventFactory.createEvent(
    maturityDate,
    EventType.MD,
    currency,
    new POF_MD_PAM(),    // ← POF attached here
    new STF_MD_PAM(),    // ← STF attached here
    businessDayAdjuster,
    contractID
));
```

When `apply()` later calls `event.eval()`, it dispatches directly to these pre-attached function objects — no further type-checking or dispatch required.

This design means adding a new contract type requires only: a new contract class, new function classes, and registration in `ContractType`. The event and function infrastructure is entirely reused.

Repository

Repository

Location

The ACTUS Technical Specifications repository is hosted on GitHub at:

`https://github.com/actusfrf/actus-techspecs`

Files

The repository contains the following files:

File	Description
`actus-techspecs.tex`	The main LaTeX source file containing the complete technical specification
`actus-techspecs.pdf`	The compiled PDF version of the specification
`bibliography.bib`	Bibliography references for the specification document
`build.sh`	Shell script to compile the LaTeX source into a PDF
`README.md`	Repository overview, build instructions, and license information
`CONTRIBUTING.md`	Guidelines for contributing to the specification
`LICENSE.md`	The full text of the Creative Commons CC-BY-SA 4.0 license
`VERSION.md`	Current version information
`.gitignore`	Standard Git ignore configuration

Version

The current release of the specification is **Version 1.1**, released on **June 8, 2020**.

License

The ACTUS Technical Specification Document is freely available and licensed under the **Creative Commons Attribution Share-Alike (CC-BY-SA) version 4.0** license.

Accessing the PDF

The latest compiled PDF is generally available at:

`https://www.actusfrf.org/algorithmic-standard`

Building from Source

The specification can be built as a PDF from the LaTeX source. Clone the repository and run the build script:

```
git clone https://github.com/actusfrf/actus-techspecs.git
cd actus-techspecs
./build.sh
```

Running `build.sh` creates the necessary configuration files and compiles the LaTeX source into a PDF version of the ACTUS Technical Specifications Document. Standard `pdflatex` tools are used for compilation.

Contributing

Contribution guidelines are described in `CONTRIBUTING.md` in the repository. The specification is maintained by the ACTUS Users Association. Issues and pull requests can be submitted through the GitHub repository.

Relationship to Other Repositories

The techspecs repository is part of the broader `actusfrf` GitHub organization, which also includes:

- `actusfrf/actus-dictionary` — the ACTUS Data Dictionary
- `actusfrf/actus-core` — the Java reference implementation
- `actusfrf/actus-core-license` — the ACTUS Core License
- `actusfrf/actus-webapp` — the ACTUS Demo App

State Space

State Space

Purpose

`StateSpace` is the contract's memory between events. It holds the current values of all analytical variables — the outstanding principal, accrued interest, the nominal rate, and more. Every `StateTransitionFunction` reads the pre-event state and writes a post-event state.

`org.actus.states.StateSpace` — `public final class` (109 lines)

All State Variables

All fields are `public` (no getters/setters). The entire class is a plain data container.

```
public final class StateSpace {
    // Core monetary values
    public double notionalPrincipal;        // Current outstanding
principal
    public double notionalPrincipal2;       // Second leg principal
(swaps, BCS)
    public double accruedInterest;          // Accrued interest not yet
paid
    public double accruedInterest2;         // Second leg accrued
interest
    public double nominalInterestRate;      // Current interest rate
    public double nominalInterestRate2;     // Second leg interest rate

    // Fee tracking
    public double feeAccrued;              // Accrued fees not yet paid

    // Interest calculation base (for ICB-based contracts)
    public double interestCalculationBaseAmount;

    // Scaling factors
    public double notionalScalingMultiplier;  // Applied to notional in
payoff
```

```
    public double interestScalingMultiplier;  // Applied to interest in
payoff

    // Redemption
    public double nextPrincipalRedemptionPayment;  // LAM/NAM/ANN next
payment

    // Key dates
    public LocalDateTime statusDate;       // Date of last state update
    public LocalDateTime maturityDate;     // Effective maturity (may
differ from CT attribute)
    public LocalDateTime terminationDate;  // Early termination date (if
exercised)
    public LocalDateTime nonPerformingDate;// Date contract first non-
performed

    // Performance status
    public ContractPerformance contractPerformance;  // Default: PF

    // Last interest period (for IPCB)
    public double lastInterestPeriod;

    // Option exercise
    public double exerciseAmount;          // Payoff at exercise
    public LocalDateTime exerciseDate;     // Date option was exercised

    // Boundary control (BCS only)
    public boolean boundaryCrossedFlag;
    public boolean boundaryMonitoringFlag;
    public boolean boundaryLeg1ActiveFlag;
    public boolean boundaryLeg2ActiveFlag;
}
```

Variable Roles

Variable	Updated by	Purpose
`notionalPrincipal`	`STF_IED`, `STF_PR`, `STF_IPCI`, `STF_RR`	Outstanding principal — the base for interest calculation
`accruedInterest`	Every `STF_IP`, `STF_FP`, `STF_RR`, `STF_SC`	Accumulated interest since last payment

Variable	Updated by	Purpose
`nominalInterestRate`	`STF_RR`, `STF_RRF`, `STF_IED`	Current rate applied to principal
`feeAccrued`	`STF_FP`, `STF_IP`	Accumulated fees since last payment
`nextPrincipalRedemptionPayment`	`STF_PRF` (ANN), `STF_IED` (LAM)	The next scheduled principal amount
`notionalScalingMultiplier`	`STF_SC`	Applied to notional in all subsequent POF calls
`interestScalingMultiplier`	`STF_SC`	Applied to interest in all subsequent POF calls
`statusDate`	Every STF	Tracks the date of the last transition
`contractPerformance`	`STF_CD`	Changes to `DF` on credit default
`nonPerformingDate`	`STF_CD`	Set when performance first degrades
`exerciseAmount` / `exerciseDate`	`STF_XD` (OPTNS)	Records option exercise
`boundaryCrossedFlag` etc.	`STF_ME` (BCS)	Controls leg switching

State Initialisation

Before the first event in `apply()` is evaluated, each contract type initialises a `StateSpace` from the contract model. The initialisation reads the contract's current snapshot from model attributes such as `statusDate`, `notionalPrincipal`, `accruedInterest`, `nominalInterestRate`, and `contractPerformance`.

For a PAM contract at inception:

```
notionalPrincipal       = model.notionalPrincipal
nominalInterestRate     = model.nominalInterestRate
accruedInterest         = model.accruedInterest  (often 0 at inception)
feeAccrued              = model.feeAccrued        (often 0)
notionalScalingMultiplier = 1.0
interestScalingMultiplier = 1.0
```

```
statusDate              = model.statusDate
contractPerformance     = model.contractPerformance  (usually PF)
```

For a contract that has already been running (mid-life state rehydration), the accrued values and current rate come from the model attributes as of the status date.

State Evolution Diagram

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
stateDiagram-v2
    [*] --> Initialised : apply() reads model attributes

    Initialised --> PostIED : STF_IED\nnotionalPrincipal set\naccruedInterest =

    PostIED --> PostIP : STF_IP\naccruedInterest = 0\nstatusDate = eventTime

    PostIP --> PostRR : STF_RR\naccruedInterest += accrual\nnominalInterestRate

    PostRR --> PostIP : next interest period

    PostIP --> PostPR : STF_PR (LAM)\nnotionalPrincipal -= redemption\nnextPrin

    PostIP --> PostSC : STF_SC\nnotionalScalingMultiplier updated\ninterestScal

    PostIP --> PostMD : STF_MD\nnotionalPrincipal = 0\naccruedInterest = 0

    PostMD --> [*]
```

Accrued Interest Accumulation

Interest accrues continuously between events. Any STF that does not pay interest must still accumulate the accrual into `accruedInterest` so it is available at the next IP event.

A typical STF that does not pay interest (e.g. `STF_RR_PAM`) updates:

```
states.accruedInterest +=
    states.nominalInterestRate
    × dayCounter.dayCountFraction(states.statusDate, time)
    × states.notionalPrincipal
    × states.interestScalingMultiplier
```

An IP event then pays the full `accruedInterest` and resets it to zero.

An IPCI (capitalisation) event adds the accrued interest to the principal instead:

```
states.notionalPrincipal += states.accruedInterest
states.accruedInterest = 0
```

`copyStateSpace`

```
public static StateSpace copyStateSpace(StateSpace original)
```

A deep-copy utility. Used where a snapshot of the state at a specific point is needed (e.g. for valuation or scenario analysis) without mutating the running state.

`toString`

Returns a formatted string of all field values, suitable for logging and test output:

```
StatusDate: 2025-01-01 | NotionalPrincipal: 1000000.0 |
NominalInterestRate: 0.05 | AccruedInterest: 0.0 | ...
```

Conventions

Conventions

The Java implementation provides four convention families, each realised as a set of interfaces and implementations in `org.actus.conventions`.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
graph TD
    subgraph daycount["org.actus.conventions.daycount"]
        DCP["DayCountConventionProvider\n(interface)"]
        DCC["DayCountCalculator\n(factory + coordinator)"]
        A360["ActualThreeSixty"]
        A365["ActualThreeSixtyFiveFixed"]
        AA["ActualActualISDA"]
        A336["ActualThreeThirtySix"]
        B252["BusinessTwoFiftyTwo"]
        E360["ThirtyEThreeSixty"]
        E360I["ThirtyEThreeSixtyISDA"]
        T28["TwentyEightThreeThirtySix"]
    end
    subgraph businessday["org.actus.conventions.businessday"]
        BDC["BusinessDayConvention\n(interface)"]
        SCC["ShiftCalcConvention\n(interface)"]
        BDA["BusinessDayAdjuster\n(coordinator)"]
        FOL["Following"]
        MF["ModifiedFollowing"]
        PRE["Preceeding"]
        MP["ModifiedPreceeding"]
        SAM["Same"]
        CS["CalcShift"]
        SC["ShiftCalc"]
    end
    subgraph eom["org.actus.conventions.endofmonth"]
        EOMC["EndOfMonthConvention\n(interface)"]
        EOMA["EndOfMonthAdjuster\n(static adjust)"]
        EOM["EndOfMonth"]
        SD["SameDay"]
    end
    subgraph role["org.actus.conventions.contractrole"]
        CRC["ContractRoleConvention\n(roleSign)"]
    end

    A360 -.-> DCP
    A365 -.-> DCP
```

```
    AA -.-> DCP
    A336 -.-> DCP
    B252 -.-> DCP
    E360 -.-> DCP
    E360I -.-> DCP
    T28 -.-> DCP
    DCC --> DCP

    FOL -.-> BDC
    MF -.-> BDC
    PRE -.-> BDC
    MP -.-> BDC
    SAM -.-> BDC
    CS -.-> SCC
    SC -.-> SCC
    BDA --> BDC
    BDA --> SCC

    EOM -.-> EOMC
    SD -.-> EOMC
    EOMA --> EOMC
```

Day Count Conventions

Interface — `DayCountConventionProvider`

```
public interface DayCountConventionProvider {
    double dayCountFraction(LocalDateTime startTime, LocalDateTime
endTime);
}
```

Factory — `DayCountCalculator`

```
// Construct from convention string + calendar
DayCountCalculator dcc = new DayCountCalculator("A/365", calendar);

// Delegate to
double fraction = dcc.dayCountFraction(startDate, endDate);
```

The constructor maps the convention string to the correct implementation class.

Implementations

Class	Convention	Formula
`ActualThreeSixty`	A/360	`actualDays / 360`
`ActualThreeSixtyFiveFixed`	A/365	`actualDays / 365`
`ActualActualISDA`	AA / A/A ISDA	Split at year boundaries; divide by 365 or 366
`ActualThreeThirtySix`	A/336	`actualDays / 336`
`BusinessTwoFiftyTwo`	B/252	`businessDays(calendar) / 252`
`ThirtyEThreeSixty`	30E/360	`(360Δy + 30Δm + Δd) / 360` with D→30
`ThirtyEThreeSixtyISDA`	30E/360 ISDA	Same + ISDA February maturity rule
`TwentyEightThreeThirtySix`	28/336	`(336Δy + 28Δm + Δd) / 336` with D→28

`ThirtyEThreeSixtyISDA` requires `setMaturityDate(LocalDateTime)` before use, because the ISDA rule preserves the exact day of a February maturity date.

`BusinessTwoFiftyTwo` depends on a `BusinessDayCalendarProvider` to count only business days.

Business Day Conventions

Two Cooperating Interfaces

```
// Shifts a non-business day to a business day
public interface BusinessDayConvention {
    LocalDateTime shift(LocalDateTime date);
}

// Decides which date to use for interest calculation
public interface ShiftCalcConvention {
    LocalDateTime shiftCalcTime(LocalDateTime date, BusinessDayConvention
bdc, BusinessDayCalendarProvider cal);
}
```

Coordinator — `BusinessDayAdjuster`

```
LocalDateTime shiftEventTime(LocalDateTime time);  // always shifts
LocalDateTime shiftCalcTime(LocalDateTime time);   // shifted or
original, per convention
```

Shift Implementations

Class	Behaviour
`Same`	No movement — `NOS`
`Following`	Advance to next business day
`ModifiedFollowing`	Advance, but back if month boundary crossed
`Preceeding`	Move back to previous business day
`ModifiedPreceeding`	Move back, but forward if month boundary crossed

Calc/Shift Implementations

Class	Behaviour
`ShiftCalc`	`shiftCalcTime` returns the shifted date — calculation follows settlement
`CalcShift`	`shiftCalcTime` returns the original date — calculation stays on contractual date

Nine Combined Conventions

Code	Shift Class	Calc Class	Meaning
NOS	`Same`	`ShiftCalc`	No adjustment at all
SCF	`Following`	`ShiftCalc`	Shift + calc both move forward
SCMF	`ModifiedFollowing`	`ShiftCalc`	Both move, stay in month
SCP	`Preceeding`	`ShiftCalc`	Both move backward
SCMP	`ModifiedPreceeding`	`ShiftCalc`	Both move back, stay in month
CSF	`Following`	`CalcShift`	Settle forward; calc on original

Code	Shift Class	Calc Class	Meaning
CSMF	`ModifiedFollowing`	`CalcShift`	Settle mod-forward; calc on original
CSP	`Preceeding`	`CalcShift`	Settle back; calc on original
CSMP	`ModifiedPreceeding`	`CalcShift`	Settle mod-back; calc on original

End-of-Month Convention

Interface

```
public interface EndOfMonthConvention {
    LocalDateTime shift(LocalDateTime date);
}
```

Implementations

Class	Behaviour
`EndOfMonth`	Snaps to the last calendar day of the month
`SameDay`	Returns the date unchanged

Coordinator — `EndOfMonthAdjuster`

```
public static LocalDateTime adjust(LocalDateTime date,
EndOfMonthConvention convention) {
    return convention.shift(date);
}
```

`EndOfMonth.shift` implementation:

```
return date.withDayOfMonth(date.toLocalDate().lengthOfMonth());
```

`ScheduleFactory` applies this adjuster to each generated date when the EOM convention is active (i.e. the cycle anchor is the last day of its month and the cycle is month-based).

Contract Role Convention

`org.actus.conventions.contractrole.ContractRoleConvention` — `public final class`

```
public static int roleSign(ContractRole role) {
    switch (role) {
        case RPA: return  1;  // Receivable principal amount — inflow
        case RPL: return -1;  // Payable principal amount — outflow
        case RFL: return  1;
        case PFL: return -1;
        case RF:  return  1;
        case PF:  return -1;
        case BUY: return  1;
        case SEL: return -1;
        ...
    }
}
```

`roleSign` is called inside virtually every `POF_*` function to flip the direction of cash flows based on the contract's role in the transaction. A lender (RPA) receives principal inflow at IED and pays out at MD; a borrower (RPL) is the mirror image.

How Conventions Flow Through the Engine

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
sequenceDiagram
    participant CT as ContractType impl
    participant SF as ScheduleFactory
    participant EF as EventFactory
    participant CE as ContractEvent
    participant POF
    participant STF

    CT->>CT: build DayCountCalculator from model
    CT->>CT: build BusinessDayAdjuster from model
    CT->>CT: build EndOfMonthConvention from model

    CT->>SF: createSchedule(anchor, end, cycle, eom)
    SF->>SF: apply EndOfMonthAdjuster to each date
    SF-->>CT: Set<LocalDateTime> rawDates
```

```
    CT->>EF: createEvents(rawDates, type, pof, stf, bda, ...)
    EF->>EF: bda.shiftEventTime(date) → eventTime
    EF-->>CT: Set<ContractEvent>

    CT->>CE: event.eval(states, model, observer, dcc, bda)
    CE->>POF: eval(..., dcc, bda) uses dcc.dayCountFraction()
    CE->>STF: eval(..., dcc, bda) uses bda.shiftCalcTime()
```

Schedule Generation

Schedule Generation

Overview

Schedule generation converts a contract's cycle parameters into an ordered set of `LocalDateTime` values that will become contract events. The core class is `ScheduleFactory`; it delegates cycle arithmetic to `CycleAdjuster` and date snapping to `EndOfMonthAdjuster`.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ScheduleFactory {
        +createSchedule(start, end, cycle, eomConvention) Set~LocalDateTime~
        +createSchedule(start, end, cycle, eomConvention, addEndTime) Set~LocalD
    }
    class CycleAdjuster {
        +CycleAdjuster(String cycle)
        +plusCycle(LocalDateTime) LocalDateTime
        +minusCycle(LocalDateTime) LocalDateTime
    }
    class CycleAdjusterProvider {
        <<interface>>
        +plusCycle(LocalDateTime) LocalDateTime
        +minusCycle(LocalDateTime) LocalDateTime
    }
    class PeriodCycleAdjuster {
        +plusCycle(LocalDateTime) LocalDateTime
        +minusCycle(LocalDateTime) LocalDateTime
    }
    class WeekdayCycleAdjuster {
        +plusCycle(LocalDateTime) LocalDateTime
        +minusCycle(LocalDateTime) LocalDateTime
    }
    CycleAdjuster --> CycleAdjusterProvider
    CycleAdjusterProvider <|.. PeriodCycleAdjuster
    CycleAdjusterProvider <|.. WeekdayCycleAdjuster
```

ScheduleFactory

`org.actus.time.ScheduleFactory` — `public final class` (177 lines)

Signatures

```
public static Set<LocalDateTime> createSchedule(
    LocalDateTime startTime,
    LocalDateTime endTime,
    String cycle,
    EndOfMonthConventionEnum endOfMonthConvention)

public static Set<LocalDateTime> createSchedule(
    LocalDateTime startTime,
    LocalDateTime endTime,
    String cycle,
    EndOfMonthConventionEnum endOfMonthConvention,
    boolean addEndTime)
```

Returns a `Set<LocalDateTime>`. Using a `Set` prevents duplicates that can arise when a generated date coincidentally equals the end date.

Algorithm

```
1. If cycle == null:
   Return {startTime} (+ endTime if addEndTime)

2. Determine if cycle is period-based (CycleUtils.isPeriod):
   - period-based → PeriodCycleAdjuster(parsePeriod(cycle))
   - weekday-based → WeekdayCycleAdjuster(parsePosition(cycle), weekday)

3. Determine EndOfMonthAdjuster:
   - If eomConvention == EOM and startTime is last day of month
     and cycle is month-based → EndOfMonth adjuster
   - else → SameDay adjuster

4. Generate dates forward from startTime:
   counter = 1
   current = startTime
   while current < endTime:
       add current to schedule
       rawNext = cycleAdjuster.plusCycle(startTime) × counter
                 (anchored to startTime, not previous date)
       current = eomAdjuster.shift(rawNext)
```

```
        counter++

5. If addEndTime: add endTime to schedule

6. Handle long stub (stub == '0'):
   If last generated date ≠ endTime and schedule.size > 2:
        remove second-to-last entry
        (merges the partial tail period into the preceding full period)

7. Return schedule
```

Key design: anchor-relative arithmetic. Each date is computed as `startTime + n × period`, not as `previousDate + period`. This eliminates cumulative rounding errors in month arithmetic (e.g. adding 1 month repeatedly from Jan 31 produces different results than computing Jan 31 + n months directly).

Stub Handling

The ACTUS cycle string includes a stub indicator as its last character (after the `L`):

- `L0` — **Long stub**: the partial period at the boundary is merged into the adjacent full period, creating one longer period. The second-to-last schedule date is removed.
- `L1` — **Short stub**: the partial period stands alone as a short period. No date removal.

CycleAdjuster

`org.actus.time.CycleAdjuster` — `public class`

The constructor inspects the cycle string and instantiates the correct delegate:

```
public CycleAdjuster(String cycle) {
    if (CycleUtils.isPeriod(cycle)) {
        this.adjuster = new
PeriodCycleAdjuster(CycleUtils.parsePeriod(cycle));
    } else {
        this.adjuster = new WeekdayCycleAdjuster(
            CycleUtils.parsePosition(cycle), weekdayFrom(cycle));
    }
}

public LocalDateTime plusCycle(LocalDateTime time) { return
adjuster.plusCycle(time); }
```

```
public LocalDateTime minusCycle(LocalDateTime time) { return
adjuster.minusCycle(time); }
```

PeriodCycleAdjuster

For cycle strings starting with `P` (ISO 8601 period format: `P[n][unit]L[stub]`). Delegates to Java's `LocalDateTime.plus(Period)` and `LocalDateTime.minus(Period)`.

WeekdayCycleAdjuster

For weekday cycle strings such as `"2Mon"` (the second Monday of each month).

```
plusCycle(date):
    targetMonth = date.plusMonths(1)
    firstWeekday = find first [weekday] in targetMonth
    result = firstWeekday + (position − 1) × 7 days
    if result.month ≠ targetMonth → result −= 7 days  (overflow guard)
    return result
```

CycleUtils

`org.actus.util.CycleUtils` — `public final class`

Method	Signature	Description
`isPeriod`	`boolean isPeriod(String cycle)`	Returns `true` if cycle starts with `'P'`
`parsePeriod`	`Period parsePeriod(String cycle)`	Parses `P[n][unit]` into a `java.time.Period`
`parsePeriod`	`Period parsePeriod(String cycle, boolean stub)`	Variant with stub awareness
`parsePosition`	`int parsePosition(String cycle)`	Extracts the ordinal from a weekday cycle string

Business Day Calendars

All three implementations are in `org.actus.time.calendar`.

Interface — `BusinessDayCalendarProvider`

```
public interface BusinessDayCalendarProvider {
    boolean isBusinessDay(LocalDate date);
    Set<LocalDate> holidays();
}
```

Implementations

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class BusinessDayCalendarProvider {
        <<interface>>
        +isBusinessDay(LocalDate) boolean
        +holidays() Set~LocalDate~
    }
    class NoHolidaysCalendar {
        +isBusinessDay(LocalDate) boolean  ← always true
        +holidays() Set~LocalDate~         ← empty set
    }
    class MondayToFridayCalendar {
        +isBusinessDay(LocalDate) boolean  ← Mon–Fri
        +holidays() Set~LocalDate~         ← empty set
    }
    class MondayToFridayWithHolidaysCalendar {
        -Set~LocalDate~ holidays
        +isBusinessDay(LocalDate) boolean  ← Mon–Fri and not in holidays
        +holidays() Set~LocalDate~
    }
    BusinessDayCalendarProvider <|.. NoHolidaysCalendar
    BusinessDayCalendarProvider <|.. MondayToFridayCalendar
    BusinessDayCalendarProvider <|.. MondayToFridayWithHolidaysCalendar
```

Class	Calendar Code	Business Days
`NoHolidaysCalendar`	NC	Every day
`MondayToFridayCalendar`	MF	Monday–Friday

Class	Calendar Code	Business Days
`MondayToFridayWithHolidaysCalendar`	MFH	Monday–Friday minus a `HashSet<LocalDate>` of holidays (O(1) lookup)

The calendar is used by `Following` / `ModifiedFollowing` / `Preceeding` / `ModifiedPreceeding` shift strategies and by `BusinessTwoFiftyTwo` day count convention.

TimeAdjuster

`org.actus.time.TimeAdjuster`

Normalises timestamps with sub-hour precision:

```
if (minutes < 30) → truncate to current hour
else              → advance to next hour
```

Used when event timestamps are loaded from systems that store time with minute or second precision.

Constants

`org.actus.util.Constants`

```
public static final Period MAX_LIFETIME     = Period.ofYears(50);
public static final Period MAX_LIFETIME_STK = Period.ofYears(10);
public static final Period MAX_LIFETIME_UMP = Period.ofYears(10);
```

These are used as `endTime` guards in `ScheduleFactory.createSchedule` when the contract model does not provide a concrete maturity date (open-ended contracts like UMP and STK).

Type System

Type System

Overview

The `org.actus.types` package contains 25 source files: 23 enumerations and the `ContractReference` value class. Every enum constant corresponds to a code string defined in the ACTUS Data Dictionary. The package has no dependencies on other `org.actus` packages; all other packages import from here.

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ContractTypeEnum { PAM ANN NAM LAM LAX CLM UMP CSH STK COM SWAPS SWPP
    class EventType { AD IED FP PR PD PRF PY PP IP IPCI CE RRF RR DV PRD MR TD
    class ContractRole { RPA RPL RFL PFL RF PF BUY SEL COL CNO UDL UDLP UDLM }
    class ContractPerformance { PF DL DQ DF MA TE }
    class ContractReference {
        +ReferenceRole referenceRole
        +ReferenceType referenceType
        +Object object
        +getObject() Object
        +getContractAttribute(String) String
        +getStateSpaceAtTimepoint(LocalDateTime, RiskFactorModelProvider) State
        +getEvent(EventType, LocalDateTime, RiskFactorModelProvider) ContractEv
    }
    ContractReference --> ReferenceRole
    ContractReference --> ReferenceType
```

ContractTypeEnum

`org.actus.types.ContractTypeEnum` — 19 values

Constant	Contract Type
`PAM`	Principal at Maturity
`ANN`	Annuity

Constant	Contract Type
`NAM`	Negative Amortizer
`LAM`	Linear Amortizer
`LAX`	Exotic Linear Amortizer
`CLM`	Call Money
`UMP`	Undefined Maturity Profile
`CSH`	Cash
`STK`	Stock
`COM`	Commodity
`SWAPS`	Swap
`SWPPV`	Swap with Performance Variation
`FXOUT`	FX Outright
`CAPFL`	Cap/Floor
`FUTUR`	Futures
`OPTNS`	Options
`CEG`	Credit Enhancement Guarantee
`CEC`	Credit Enhancement Collateral
`BCS`	Boundary Controlled Switch

These 19 values map one-to-one to the 19 contract implementation classes in `org.actus.contracts`. `ContractType` (the public dispatcher) uses `ContractTypeEnum.valueOf(model.getAs("contractType"))` to route calls.

EventType

`org.actus.types.EventType` — 30 values

Constant	Meaning
`AD`	Analysis Date — state snapshot with no cash flow
`IED`	Initial Exchange Date — contract inception; principal disbursed
`FP`	Fee Payment
`PR`	Principal Redemption
`PD`	Principal Drawing (UMP/CLM)
`PRF`	Principal Payment at Fixed Schedule
`PY`	Penalty Payment
`PP`	Principal Prepayment
`IP`	Interest Payment
`IPCI`	Interest Capitalisation — accrued interest added to principal
`CE`	Credit Event
`RRF`	Rate Reset Fixed — interest rate reset to a predefined value
`RR`	Rate Reset Variable — interest rate reset via market lookup
`DV`	Dividend Payment
`PRD`	Purchase Date
`MR`	Margin Call
`TD`	Termination Date
`SC`	Scaling Index Revision
`IPCB`	Interest Calculation Base Revision
`MD`	Maturity Date
`XD`	Execution Date (options)
`STD`	Settlement Date
`PI`	Principal Increase
`IPFX`	Interest Payment Fixed Leg

Constant	Meaning
`IPFL`	Interest Payment Floating Leg
`ME`	Monitoring Event
`AFD`	Accrual / Fee Draw (internal scheduling)

`EventType` values appear as the second element of the `ContractEvent` tuple and control which `POF_[EventType]_[ContractType]` and `STF_[EventType]_[ContractType]` functions are invoked during evaluation.

ContractRole

`org.actus.types.ContractRole` — 13 values

Constant	Description
`RPA`	Real Party Asset — the contract is held as an asset
`RPL`	Real Party Liability — the contract is held as a liability
`RFL`	Real Party Fixed Leg of a swap
`PFL`	Pay Fixed Leg of a swap
`RF`	Reference Fixed Leg
`PF`	Pay Floating Leg
`BUY`	Buyer (options, futures)
`SEL`	Seller (options, futures)
`COL`	Collateral provider
`CNO`	Collateral receiver (counterparty nominal obligation)
`UDL`	Underlying
`UDLP`	Underlying — put side
`UDLM`	Underlying — mortgage side

`ContractRoleConvention.roleSign(ContractRole)` maps `RPA`, `BUY`, `RFL`, `COL`, `UDLP` to `+1.0` and the remaining roles to `-1.0`. This sign is multiplied into every cash flow so that assets and liabilities produce cash flows of opposite sign from the same formula.

ContractPerformance

`org.actus.types.ContractPerformance` — 6 values

Constant	Meaning
`PF`	Performing — contract is current
`DL`	Delayed — payment overdue but within grace period
`DQ`	Delinquent — past grace period, not yet defaulted
`DF`	Default — contract is in default
`MA`	Matured — contract has reached its maturity date
`TE`	Terminated — contract has been terminated early

Stored in `StateSpace.contractPerformance`. State transitions driven by `CE` (credit event) events update this field. Many POF implementations check the performance status to determine whether a penalty applies.

BusinessDayConventionEnum

`org.actus.types.BusinessDayConventionEnum` — 9 values

Constant	Convention
`NOS`	No shift — use scheduled date regardless
`SCF`	Shift/Calculate Following
`SCMF`	Shift/Calculate Modified Following
`CSF`	Calculate/Shift Following
`CSMF`	Calculate/Shift Modified Following

Constant	Convention
`SCP`	Shift/Calculate Preceding
`SCMP`	Shift/Calculate Modified Preceding
`CSP`	Calculate/Shift Preceding
`CSMP`	Calculate/Shift Modified Preceding

The `SC` vs `CS` prefix determines whether the date is first shifted to a business day and then used for calculation, or first used for calculation and then shifted. The Java conventions package maps each code to a `(ShiftCalcConvention, BusinessDayAdjusterProvider)` pair.

EndOfMonthConventionEnum

`org.actus.types.EndOfMonthConventionEnum` — 2 values

Constant	Behaviour
`SD`	Same Day — generated dates remain on the same calendar day as the anchor
`EOM`	End of Month — when the anchor falls on the last day of a month and the cycle is month-based, subsequent dates snap to the last day of each target month

---

Calendar

`org.actus.types.Calendar` — 3 values

Constant	Calendar Code	Business Days
`NC`	No Calendar	Every calendar day
`MF`	Monday–Friday	Monday through Friday only
`MFH`	Monday–Friday + Holidays	Monday through Friday, excluding a provided holiday set

Maps to `NoHolidaysCalendar`, `MondayToFridayCalendar`, and `MondayToFridayWithHolidaysCalendar` in `org.actus.time.calendar`.

FeeBasis

`org.actus.types.FeeBasis` — 2 values

Constant	Meaning
`A`	Absolute — fee is a fixed monetary amount
`N`	Notional — fee is expressed as a fraction of the current notional principal

Controls the fee payoff formula in `POF_FP_*` implementations.

ScalingEffect

`org.actus.types.ScalingEffect` — 4 values

Constant	Interest Scaled	Notional Scaled
`OOO`	No	No
`IOO`	Yes	No
`ONO`	No	Yes
`INO`	Yes	Yes

The two-character suffix encodes interest (`I` / `O`) and notional (`N` / `O`) scaling independently. When active, `StateSpace.interestScalingMultiplier` and `StateSpace.notionalScalingMultiplier` are applied in the relevant POF and STF.

PenaltyType

`org.actus.types.PenaltyType` — 4 values

Constant	Penalty Basis
`N`	No penalty
`A`	Absolute fixed amount

Constant	Penalty Basis
`R`	Rate — fraction of outstanding notional
`I`	Current interest amount

Used in `POF_PP_PAM` and related prepayment payoff functions.

InterestCalculationBase

`org.actus.types.InterestCalculationBase` — 3 values

Constant	Base for Accrual
`NT`	Notional — accrue on current notional principal
`NTIED`	Notional at IED — accrue on original notional at inception
`NTL`	Notional at Last reset — accrue on notional at last IPCB event

Stored in `StateSpace.interestCalculationBaseAmount`. The `IPCB` event refreshes this field when the convention requires it.

OptionType

`org.actus.types.OptionType` — 3 values

Constant	Option
`C`	Call
`P`	Put
`CP`	Call or Put (chooser)

OptionExerciseType

`org.actus.types.OptionExerciseType` — 3 values

Constant	Exercise Style
`E`	European — exercise only at expiry
`B`	Bermudan — exercise on specified dates
`A`	American — exercise any time up to expiry

DeliverySettlement

`org.actus.types.DeliverySettlement` — 2 values

Constant	Meaning
`S`	Settlement — cash settlement at expiry
`D`	Delivery — physical delivery of underlying

ClearingHouse

`org.actus.types.ClearingHouse` — 2 values

Constant	Meaning
`Y`	Yes — cleared through a central counterparty
`N`	No — bilateral OTC

Seniority

`org.actus.types.Seniority` — 2 values

Constant	Meaning
`S`	Senior
`J`	Junior (subordinated)

GuaranteedExposure

`org.actus.types.GuaranteedExposure` — 3 values

Constant	Exposure Basis
`NO`	Notional — exposure is the notional principal
`NI`	Notional plus Interest — exposure includes accrued interest
`MV`	Market Value — exposure is the current market value

Used in the credit enhancement contract types (`CEG`, `CEC`).

CreditEventTypeCovered

`org.actus.types.CreditEventTypeCovered` — 3 values

Constant	Covered Event
`DL`	Delayed payment
`DQ`	Delinquency
`DF`	Default

PrepaymentEffect

`org.actus.types.PrepaymentEffect` — 3 values

Constant	Effect of Prepayment
`N`	No effect — schedule unchanged
`A`	Reduce Annuity — payment amount decreases
`M`	Reduce Maturity — maturity date moves earlier

CyclePointOfInterestPayment

`org.actus.types.CyclePointOfInterestPayment` — 2 values

Constant	Payment Timing
`B`	Beginning of cycle — interest paid at the start of each period
`E`	End of cycle — interest paid at the end of each period

CyclePointOfRateReset

`org.actus.types.CyclePointOfRateReset` — 2 values

Constant	Reset Timing
`B`	Beginning of cycle — rate is fixed at the start of each period
`E`	End of cycle — rate is fixed at the end of each period

ReferenceRole

`org.actus.types.ReferenceRole` — 6 values

Constant	Role
`UDL`	Underlying asset or contract
`FIL`	First leg of a structured product
`SEL`	Second leg of a structured product
`COVE`	Covered entity (protection buyer)
`COVI`	Covering entity (protection seller)
`externalReferenceIndex`	External reference index (market rate source)

ReferenceType

`org.actus.types.ReferenceType` — 5 values

Constant	Reference Resolves To
`CNT`	Contract — another `ContractModel` embedded inline
`CID`	Contract ID — a reference by string ID to be resolved at runtime
`MOC`	Market Object Code — a key into the `RiskFactorModelProvider`
`EID`	Entity ID — a counterparty or legal entity identifier
`CST`	Constant — a fixed numerical constant

ContractReference

`org.actus.types.ContractReference` — `public class`

`ContractReference` wraps a reference to an underlying contract, a market data series, or an external entity. It resolves the reference at evaluation time through a `RiskFactorModelProvider`.

Fields

Field	Type	Description
`referenceRole`	`ReferenceRole`	How this reference is used in the parent contract
`referenceType`	`ReferenceType`	What the reference points to
`object`	`Object`	The resolved reference (a `ContractModel`, a market object code string, or a numeric constant)

Constructor

```
public ContractReference(Map<String, Object> attributes, ContractRole
contractRole)
```

Parses the `contractStructure` entry from a contract's attributes map. Dispatches on `referenceType`:

- `CNT` → recursively constructs a nested `ContractModel`
- `CID` → stores the contract ID string for deferred resolution
- `MOC` → stores the market object code string
- `EID` → stores the entity ID string
- `CST` → stores the constant value as a `Double`

Key Methods

```
Object getObject()
```

Returns the wrapped reference value.

```
String getContractAttribute(String contractAttribute)
```

When the reference is `CNT` or `CID`, retrieves a named attribute from the referenced contract (e.g. `"marketObjectCode"`).

```
StateSpace getStateSpaceAtTimepoint(LocalDateTime time,
RiskFactorModelProvider observer)
```

Evaluates the referenced contract up to `time` and returns its `StateSpace`. Used by credit enhancement contract types to observe the current state of the covered obligation.

```
ContractEvent getEvent(EventType eventType, LocalDateTime time,
RiskFactorModelProvider observer)
```

Retrieves a specific event from the referenced contract's evaluated schedule. Used when one contract's payoff depends on a specific event of an underlying contract.

Usage in Contract Types

`ContractReference` is used in structured product types where one contract wraps or references another:

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
flowchart LR
    CEG["CEG\n(Credit Enhancement Guarantee)"]
    CEC["CEG\n(Credit Enhancement Collateral)"]
    SWAPS["SWAPS\n(Swap)"]
    OPTNS["OPTNS\n(Options)"]
    REF["ContractReference\n(resolved at eval time)"]
    RFM["RiskFactorModelProvider"]

    CEG --> REF
    CEC --> REF
    SWAPS --> REF
    OPTNS --> REF
    REF --> RFM
```

The parent contract's `apply()` method calls `getStateSpaceAtTimepoint()` or `getEvent()` on each `ContractReference` to obtain the state or events of the underlying at the relevant evaluation timestamps.

Test Infrastructure

Test Infrastructure

Overview

The test suite validates each of the 19 contract types against JSON-encoded expected results. Tests are JUnit 5 dynamic tests — each JSON file can contain dozens or hundreds of independent test cases that the framework discovers and runs automatically.

```
test/
└── java/
    └── org/
        └── actus/
            ├── contracts/          ← 45 test classes (one or more per
contract type)
            └── testutils/          ← 7 utility classes
```

Test Utility Classes

Class Diagram

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
classDiagram
    class ContractTestUtils {
        +createModel(Map) ContractModel
        +createObserver(List~ObservedDataSet~) DataObserver
        +createObserver(List~ObservedDataSet~, List~ContractEvent~) DataObserve
        +readTests(String file) Map~String,TestData~
        +readObservedEvents(List~ObservedEvent~, ContractModel) List~ContractEv
    }
    class DataObserver {
        -HashMap~String,HashMap~LocalDateTime,Double~~ multiSeries
        -HashMap~String,List~ContractEvent~~ eventsObserved
        +keys() Set~String~
        +add(String, HashMap) void
        +stateAt(id, time, states, attrs, isMarket) double
        +setEventsObserved(List~ContractEvent~) void
```

```
        +events(ContractModelProvider) Set~ContractEvent~
    }
    class TestData {
        +identifier String
        +terms Map~String,Object~
        +to String
        +dataObserved Map~String,ObservedDataSet~
        +eventsObserved List~ObservedEvent~
        +results List~ResultSet~
    }
    class ObservedDataSet {
        +identifier String
        +data List~ObservedDataPoint~
    }
    class ObservedDataPoint {
        +timestamp String
        +value Double
    }
    class ObservedEvent {
        +time String
        +type String
        +value double
        +contractId String
        +states StateSpace
    }
    class ResultSet {
        +eventDate LocalDateTime
        +eventType EventType
        +payoff Double
        +notionalPrincipal Double
        +nominalInterestRate Double
        +accruedInterest Double
        +currency String
        +roundTo(int) void
        +equals(Object) boolean
    }
    ContractTestUtils --> DataObserver
    ContractTestUtils --> TestData
    TestData --> ObservedDataSet
    TestData --> ObservedEvent
    TestData --> ResultSet
    ObservedDataSet --> ObservedDataPoint
    DataObserver ..|> RiskFactorModelProvider
```

ContractTestUtils

`org.actus.testutils.ContractTestUtils` — `public final class`

The central factory class. All test classes use its static methods to convert JSON data into the runtime objects required by `ContractType.schedule()` and `ContractType.apply()`.

Methods

`readTests`

```
public static Map<String, TestData> readTests(String file)
```

Reads a JSON test file from the classpath using Jackson's `ObjectMapper`. Returns a `Map` keyed by test case identifier (e.g. `"pam001"`). The Jackson `ObjectMapper` is configured to deserialise `TestData` and its nested structures automatically.

`createModel`

```
public static ContractModel createModel(Map<String, Object> data)
```

Converts the raw `terms` map from a `TestData` instance into a `ContractModel`. Special handling:

- `contractStructure` entries are parsed into `ContractReference` objects
- List values (e.g. arrays of schedule dates) are converted to comma-separated strings before being stored, matching the `ContractModel`'s internal string representation for cycle and schedule attributes

`createObserver` (two overloads)

```
public static DataObserver createObserver(List<ObservedDataSet> data)
public static DataObserver createObserver(List<ObservedDataSet> data,
                                          List<ContractEvent>
observedEvents)
```

Builds a `DataObserver` from the `dataObserved` list. Each `ObservedDataSet` becomes a `HashMap<LocalDateTime, Double>` keyed by `symbol` inside `DataObserver.multiSeries`. The second overload additionally registers pre-observed contract events (for credit enhancement tests where the covered obligation's events must be available to the covering contract).

`readObservedEvents`

```
public static List<ContractEvent> readObservedEvents(
    List<ObservedEvent> eventsObserved, ContractModel terms)
```

Converts `ObservedEvent` objects (raw JSON) into fully formed `ContractEvent` objects by matching event type codes and attaching the appropriate state snapshots.

DataObserver

`org.actus.testutils.DataObserver` — `implements RiskFactorModelProvider`

Provides market data and pre-observed contract events to the contract evaluation engine. This is the test-environment implementation of the `RiskFactorModelProvider` interface.

Fields

```
private HashMap<String, HashMap<LocalDateTime, Double>> multiSeries;
private HashMap<String, List<ContractEvent>> eventsObserved;
```

`multiSeries` is a two-level map: outer key is the market object code (e.g. `"USD_LIBOR_3M"`), inner key is a `LocalDateTime`, value is the observed rate or price. Lookup is O(1) for both dimensions.

`stateAt` Implementation

```
public double stateAt(String id, LocalDateTime time,
                      StateSpace contractStates,
                      ContractModelProvider contractAttributes,
                      boolean isMarket)
```

Returns the value of market object `id` at `time`. If the exact timestamp is not present, the implementation returns the value at the latest timestamp before `time` (flat forward / last-observation-carried-forward interpolation). If `id` is not found, returns `0.0`.

`events` Implementation

```
public Set<ContractEvent> events(ContractModelProvider model)
```

Returns the set of pre-observed events associated with the contract identified by `model.getAs("contractID")`. Used by credit enhancement contract types that need to observe events from the referenced covered obligation.

TestData

`org.actus.testutils.TestData` — Jackson-deserialised test case wrapper

Field	Type	Description
`identifier`	`String`	Test case ID, e.g. `"pam001"`
`terms`	`Map<String, Object>`	Contract terms — raw JSON key/value pairs
`to`	`String`	Analysis horizon date as ISO-8601 string; empty string means use maturity date
`dataObserved`	`Map<String, ObservedDataSet>`	Market data time series keyed by symbol
`eventsObserved`	`List<ObservedEvent>`	Pre-observed events (e.g. credit events on a covered contract)
`results`	`List<ResultSet>`	Expected output — one entry per expected contract event

---

ObservedDataSet and ObservedDataPoint

`org.actus.testutils.ObservedDataSet` wraps a labelled time series:

```
String identifier;              // market object code, e.g.
"USD_LIBOR_3M"
List<ObservedDataPoint> data;   // ordered list of (timestamp, value)
pairs
```

`org.actus.testutils.ObservedDataPoint` is a two-field struct:

```
String timestamp;   // ISO-8601 datetime string
Double value;       // observed rate, price, or index value
```

ObservedEvent

`org.actus.testutils.ObservedEvent` — raw JSON representation of a pre-observed event

Field	Type	Description
`time`	`String`	ISO-8601 datetime of the event
`type`	`String`	EventType code, e.g. `"CE"`
`value`	`double`	Numeric value associated with the event
`contractId`	`String`	ID of the contract this event belongs to
`states`	`StateSpace`	Contract state snapshot at the event time

ResultSet

`org.actus.testutils.ResultSet` — expected output record for a single contract event

All Fields

Field	Type	Description
`eventDate`	`LocalDateTime`	Scheduled event date
`exerciseDate`	`LocalDateTime`	Option exercise date (OPTNS only)
`eventType`	`EventType`	Event type code
`currency`	`String`	ISO 4217 currency code
`payoff`	`Double`	Cash flow amount
`accruedInterest`	`Double`	Accrued interest after event

Field	Type	Description
`accruedInterest2`	`Double`	Secondary accrued interest (swap legs)
`exerciseAmount`	`Double`	Option exercise amount
`feeAccrued`	`Double`	Accrued fee after event
`interestCalculationBaseAmount`	`Double`	Active accrual base
`interestScalingMultiplier`	`Double`	Interest scaling factor
`nextPrincipalRedemptionPayment`	`Double`	Next scheduled principal repayment
`nominalInterestRate`	`Double`	Current interest rate
`nominalInterestRate2`	`Double`	Secondary rate (swap floating leg)
`notionalPrincipal`	`Double`	Outstanding notional after event
`notionalPrincipal2`	`Double`	Secondary notional (structured products)
`notionalScalingMultiplier`	`Double`	Notional scaling factor

Key Methods

`roundTo`

```
public void roundTo(int decimals)
```

Rounds all non-null `Double` fields to `decimals` places using `BigDecimal.setScale(decimals, RoundingMode.HALF_UP)`. Both the computed and expected `ResultSet` are rounded to 10 decimal places before comparison to eliminate floating-point noise.

`setValues` and `equals`

`setValues()` populates a generic `HashMap<String, String>` from all non-null object fields using reflection. `equals(Object o)` compares `ResultSet` instances by comparing their `values` maps, so the assertion `assertArrayEquals(expected, computed)` compares all populated fields regardless of which fields the test case specifies.

JUnit 5 Dynamic Test Pattern

Every contract test class follows the same structure:

```
@TestFactory
public Stream<DynamicTest> test() throws Exception {
    // 1. Read JSON
    Map<String, TestData> tests = ContractTestUtils.readTests("actus-
tests-pam.json");

    return tests.entrySet().stream().map(entry -> {
        String testId = entry.getKey();
        TestData testData = entry.getValue();

        return DynamicTest.dynamicTest("Test: " + testId, () -> {

            // 2. Build model and observer
            ContractModel terms =
ContractTestUtils.createModel(testData.getTerms());
            DataObserver observer = ContractTestUtils.createObserver(
                testData.getDataObserved(), ...);

            // 3. Determine analysis horizon
            LocalDateTime to = testData.getTo().isEmpty()
                ? terms.getAs("maturityDate")
                : LocalDateTime.parse(testData.getTo());

            // 4. Schedule and apply
            ArrayList<ContractEvent> schedule = ContractType.schedule(to,
terms);
            schedule = ContractType.apply(schedule, terms, observer);

            // 5. Extract computed results
            List<ResultSet> computed = schedule.stream()
                .map(e -> {
                    ResultSet rs = new ResultSet();
                    rs.setEventDate(e.eventTime());
                    rs.setEventType(e.eventType());
                    rs.setPayoff(e.payoff());
                    // ... map remaining state fields from e.states() ...
                    rs.setValues();
                    return rs;
                }).collect(Collectors.toList());

            // 6. Extract expected results and round both sides
```

```
            List<ResultSet> expected = testData.getResults();
            computed.forEach(r -> r.roundTo(10));
            expected.forEach(r -> r.roundTo(10));

            // 7. Assert
            Assertions.assertArrayEquals(
                expected.toArray(), computed.toArray());
        });
    });
}
```

The `@TestFactory` method returns a `Stream<DynamicTest>`, so JUnit 5 reports each test case as a separately named test in the test runner. A single JSON file with 500 test cases produces 500 individually trackable test results.

Test Flow Diagram

DIAGRAM (RENDERED INTERACTIVELY IN BROWSER)

```
sequenceDiagram
    participant JUnit
    participant TestClass
    participant CTU as ContractTestUtils
    participant CT as ContractType
    participant DO as DataObserver

    JUnit->>TestClass: invoke @TestFactory
    TestClass->>CTU: readTests(jsonFile)
    CTU-->>TestClass: Map<id, TestData>
    loop for each test case
        TestClass->>CTU: createModel(terms)
        CTU-->>TestClass: ContractModel
        TestClass->>CTU: createObserver(dataObserved)
        CTU-->>TestClass: DataObserver
        TestClass->>CT: schedule(to, model)
        CT-->>TestClass: ArrayList<ContractEvent>
        TestClass->>CT: apply(events, model, observer)
        CT->>DO: stateAt(id, time, ...)
        DO-->>CT: double value
        CT-->>TestClass: ArrayList<ContractEvent> with payoffs
        TestClass->>JUnit: DynamicTest (expected vs computed)
    end
    JUnit->>JUnit: run each DynamicTest
```

JSON Test File Format

Each JSON file contains a top-level object whose keys are test case identifiers. Each value is a `TestData` object.

Minimal Example (CSH contract)

```
{
  "csh01": {
    "identifier": "csh01",
    "terms": {
      "contractType": "CSH",
      "statusDate": "2015-07-15T00:00:00",
      "contractRole": "RPA",
      "contractID": "csh01",
      "currency": "CHF",
      "notionalPrincipal": "1000"
    },
    "to": "2015-08-15T00:00:00",
    "dataObserved": {},
    "eventsObserved": [
      {
        "time": "2015-07-30T00:00:00",
        "type": "AD",
        "value": 0
      }
    ],
    "results": [
      {
        "eventDate": "2015-07-30T00:00",
        "eventType": "AD",
        "payoff": 0,
        "notionalPrincipal": 1000,
        "nominalInterestRate": 0,
        "accruedInterest": 0,
        "currency": "CHF"
      }
    ]
  }
}
```

Terms Object

Contains ACTUS Data Dictionary attribute names as keys and their values as strings or arrays:

```
{
  "contractType": "PAM",
  "statusDate": "2012-12-31T00:00:00",
  "contractRole": "RPA",
  "contractID": "pam001",
  "currency": "USD",
  "initialExchangeDate": "2013-01-01T00:00:00",
  "maturityDate": "2016-01-01T00:00:00",
  "notionalPrincipal": "10000",
  "nominalInterestRate": "0.05",
  "cycleOfInterestPayment": "P1YL1",
  "dayCountConvention": "A365",
  "businessDayConvention": "NOS"
}
```

dataObserved Object

Maps market object codes to time series. Empty `{}` for contracts with no market dependencies (fixed-rate, no scaling):

```
{
  "dataObserved": {
    "USD_LIBOR_3M": {
      "identifier": "USD_LIBOR_3M",
      "data": [
        { "timestamp": "2013-01-01T00:00:00", "value": 0.02 },
        { "timestamp": "2014-01-01T00:00:00", "value": 0.025 },
        { "timestamp": "2015-01-01T00:00:00", "value": 0.03 }
      ]
    }
  }
}
```

results Array

Each element captures a single expected contract event:

```
{
  "eventDate": "2014-01-01T00:00",
  "eventType": "IP",
  "payoff": 500.00,
  "notionalPrincipal": 10000,
  "nominalInterestRate": 0.05,
  "accruedInterest": 0,
```

```
  "currency": "USD"
}
```

Only fields that the test case intends to assert need to be present. `ResultSet.equals()` compares only non-null values, so a result entry that omits `feeAccrued` will not fail if the computed value is non-zero.

Test JSON Data Files

All 20 files reside in the test resources classpath (`src/test/resources/`).

File	Approx. Lines	Approx. Size	Contract Type(s)
`actus-tests-ad0.json`	434	13 KB	AD (Analysis Date — all contract types)
`actus-tests-pam.json`	5 676	160 KB	PAM — Plain Amortizer
`actus-tests-ann.json`	12 753	390 KB	ANN — Annuity
`actus-tests-nam.json`	9 669	310 KB	NAM — Negative Amortizer
`actus-tests-lam.json`	10 212	360 KB	LAM — Linear Amortizer
`actus-tests-lax.json`	3 588	122 KB	LAX — Exotic Linear Amortizer
`actus-tests-clm.json`	1 756	63 KB	CLM — Call Money
`actus-tests-ump.json`	812	28 KB	UMP — Undefined Maturity Profile
`actus-tests-csh.json`	135	3.6 KB	CSH — Cash
`actus-tests-stk.json`	1 258	42 KB	STK — Stock
	160	4.7 KB	COM — Commodity

File	Approx. Lines	Approx. Size	Contract Type(s)
`actus-tests-com.json`			
`actus-tests-swaps.json`	4 172	137 KB	SWAPS — Swap
`actus-tests-swppv.json`	2 597	91 KB	SWPPV — Swap with Performance Variation
`actus-tests-fxout.json`	574	18 KB	FXOUT — FX Outright
`actus-tests-capfl.json`	380	11 KB	CAPFL — Cap/Floor
`actus-tests-futur.json`	1 411	46 KB	FUTUR — Futures
`actus-tests-optns.json`	2 369	77 KB	OPTNS — Options
`actus-tests-ceg.json`	1 157	42 KB	CEG — Credit Enhancement Guarantee
`actus-tests-cec.json`	1 544	60 KB	CEC — Credit Enhancement Collateral
`actus-tests-bcs.json`	1 334	32 KB	BCS — Boundary Controlled Switch

The largest files (`actus-tests-ann.json` at ~390 KB and `actus-tests-lam.json` at ~360 KB) reflect the complexity of amortising contract types, which require testing many combinations of rate reset cycles, redemption schedules, day count conventions, and stub configurations.

Test Count and Coverage

The 45 test classes and 7 test utility classes together with the 20 JSON files cover all 19 contract types. Test cases exercise:

- Normal contract lifecycle (IED through MD)
- Rate reset events (RR, RRF) with flat, upward, and downward rate scenarios

- Scaling events (SC) with combined interest and notional scaling
- Prepayment events (PP) with all three `PrepaymentEffect` values
- Credit events (CE) on CEG and CEC contracts with all covered event types
- End-of-month convention behaviour for month-end anchor dates
- Business day shifting for each of the nine `BusinessDayConventionEnum` values
- Long and short stub periods at both the leading and trailing end of a schedule
- Open-ended contracts (UMP, STK) evaluated against the `MAX_LIFETIME` guard